

Livre de Soutenance

Conception et réalisation d'une application de gestion
scolaire hors-ligne avec Laravel, Electron et SQLite

Présenté par :

Tojo Nambinina RANDRIAMIFALY

Encadré par :

Nom de l'encadrant

Parcours : Communication en Audiovisuelle et Numérique (CAN)

Spécialisation : Développement Web (après L3)

Année universitaire : 2025–2026

Remerciements

En premier lieu, nous remercions Dieu tout-puissant de nous avoir donné la force, la patience et la détermination nécessaires pour mener à bien ce projet.

Nous tenons à exprimer notre profonde gratitude à notre encadrant pour ses conseils précieux, sa disponibilité et son accompagnement tout au long de ce travail. Sa confiance et ses orientations éclairées ont grandement contribué à la réussite de ce projet.

Nous remercions chaleureusement les membres du jury qui nous font l'honneur d'évaluer ce travail.

Nos sincères remerciements vont également à toutes les personnes qui ont participé de près ou de loin à la réalisation de ce projet, en particulier les enseignants et le personnel administratif des établissements scolaires qui nous ont accordé de leur temps pour les entretiens et les tests.

Enfin, nous exprimons notre reconnaissance à nos familles et à nos proches pour leur soutien indéfectible et leurs encouragements tout au long de notre parcours universitaire.

Dédicaces

Je dédie ce modeste travail à :

Mes chers parents, pour leur amour inconditionnel, leurs sacrifices et leur soutien permanent tout au long de mes études.

Mes frères et sœurs, pour leur encouragement et leur présence à mes côtés.

Tous mes amis et camarades de promotion, pour les moments partagés et l'entraide mutuelle.

Tous ceux qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Table des matières

Remerciements	2
Dédicaces	3
Introduction générale	11
1 État de l’art et technologies utilisées	13
1.1 Introduction	13
1.2 Architecture logicielle	13
1.2.1 Définition	13
1.2.2 Le modèle 4+1 vues de Philippe Kruchten	13
1.2.3 Le patron d’architecture MVC (Modèle-Vue-Contrôleur)	14
1.3 Architectures distribuées	15
1.3.1 Définition	15
1.3.2 Types d’architectures distribuées	15
1.4 Modèle Client/Serveur	15
1.4.1 Définition	15
1.4.2 Principe de fonctionnement	15
1.4.3 Architecture à 2 niveaux (2-tiers)	16
1.4.4 Architecture à 3 niveaux (3-tiers)	16
1.4.5 Architecture multi-niveaux (n-tiers)	17
1.4.6 Middleware	17
1.5 Internet et le Web	17
1.5.1 Internet	17
1.5.2 Intranet et Extranet	18
1.5.3 Le World Wide Web	18
1.5.4 Le protocole HTTP	18
1.5.5 REST API	18
1.6 Technologies de développement web	18
1.6.1 PHP	18
1.6.2 Laravel	19
1.6.3 Comparaison des frameworks PHP	20

1.6.4	Node.js et npm	20
1.6.5	Vite	20
1.6.6	TailwindCSS	21
1.7	Systèmes de Gestion de Base de Données (SGBD)	21
1.7.1	Définition	21
1.7.2	SQLite	21
1.7.3	MySQL/MariaDB	21
1.8	Applications de bureau hybrides	22
1.8.1	Electron	22
1.8.2	electron-builder	22
1.8.3	Capacitor	23
1.9	Conclusion	23
2	Architecture et technologies de Novaskol	24
2.1	Introduction	24
2.2	Présentation du projet Novaskol	24
2.2.1	Contexte et objectifs	24
2.2.2	Spécifications techniques	25
2.3	Architecture globale du système	25
2.3.1	Vue d'ensemble	25
2.3.2	L'application de bureau Electron (Novaskol Desktop)	26
2.3.3	La synchronisation multi-appareils	28
2.3.4	Modes de déploiement	28
2.4	Technologies et outils utilisés	29
2.4.1	Backend	29
2.4.2	Frontend	29
2.4.3	Desktop	29
2.4.4	Mobile	30
2.5	Architecture détaillée de la base de données	30
2.5.1	Présentation	30
2.5.2	Liste des tables principales	30
2.5.3	Dictionnaire des données	31
2.5.4	Relations principales	33
2.5.5	Système de synchronisation	33
2.6	Modules et fonctionnalités	33
2.6.1	Administration générale	33
2.6.2	Gestion des inscriptions	34
2.6.3	Gestion pédagogique	34
2.6.4	Communication	34

2.6.5	Gestion des ressources humaines	35
2.6.6	Gestion financière	35
2.6.7	Portail parents	35
2.6.8	Système	35
2.7	Conclusion	35
3	Analyse et conception	37
3.1	Introduction	37
3.2	Analyse des besoins	37
3.2.1	Besoins fonctionnels	37
3.2.2	Besoins non fonctionnels	38
3.3	Identification des acteurs	39
3.3.1	Correspondance acteurs-tâches	39
3.4	Diagrammes de cas d'utilisation	39
3.4.1	Diagramme de contexte	39
3.4.2	Cas d'utilisation par acteur	40
3.4.3	Diagramme de cas d'utilisation – Authentification	41
3.4.4	Diagramme de cas d'utilisation – Saisie des notes	41
3.5	Diagrammes de séquence	42
3.5.1	Diagramme de séquence – Authentification	42
3.5.2	Diagramme de séquence – Saisie des notes	42
3.5.3	Diagramme de séquence – Génération de bulletin	43
3.5.4	Diagramme de séquence – Synchronisation	44
3.6	Diagrammes d'activité	44
3.6.1	Diagramme d'activité – Authentification	44
3.6.2	Diagramme d'activité – Saisie des notes	45
3.7	Modèle conceptuel de données	47
3.7.1	Diagramme de classe global	47
3.7.2	Relations principales	48
3.7.3	Schéma relationnel simplifié	48
3.8	Conclusion	50
4	Réalisation	51
4.1	Introduction	51
4.2	Environnement de développement	51
4.2.1	Outils de développement	51
4.2.2	Structure du projet	51
4.2.3	Processus de développement	52
4.3	Implémentation du backend Laravel	53
4.3.1	Architecture des contrôleurs	53

4.3.2	Système d'authentification	53
4.3.3	Middleware de contrôle d'accès	54
4.3.4	Services métier	54
4.3.5	Génération de PDF	55
4.4	Implémentation du frontend	55
4.4.1	Architecture des vues	55
4.4.2	Design et thème	55
4.4.3	Composants d'interface	55
4.4.4	Internationalisation	55
4.5	Application de bureau Electron	56
4.5.1	Fichiers clés	56
4.5.2	Processus principal (main.cjs)	56
4.5.3	Fonctionnalités du processus principal	57
4.5.4	Pont IPC	57
4.5.5	Configuration electron-builder	57
4.6	Application mobile companion (Novaskol Connecte)	58
4.6.1	Architecture	58
4.6.2	Fonctionnalités	58
4.7	Application de bureau Connecte Desktop	59
4.7.1	Architecture	59
4.7.2	Différences avec la version mobile	59
4.8	Processus de packaging et déploiement	59
4.8.1	Application de bureau (Electron + electron-builder)	59
4.8.2	PHP Runtime portable	59
4.8.3	Installateur NSIS	60
4.8.4	Modes de déploiement	60
4.9	Interfaces principales	60
4.9.1	Page d'authentification	60
4.9.2	Tableau de bord	60
4.9.3	Saisie des notes	61
4.9.4	Génération de bulletins	61
4.9.5	Emploi du temps	61
4.9.6	Messagerie interne	61
4.9.7	Gestion des présences par QR code	61
4.10	Conclusion	61
	Conclusion générale	63
	Ouvrages et articles	65
	Documentation technique	65

Références web	66
A Généralités sur les réseaux informatiques	67
A.1 Définition d'un réseau	67
A.2 Intérêts d'un réseau informatique	67
A.3 Classification des réseaux	67
A.4 Topologies de réseau	67
A.5 Modèle OSI	68
A.6 Protocole TCP/IP	68
B UML – Unified Modeling Language	69
B.1 Définition	69
B.2 Diagrammes UML utilisés dans Novaskol	69
B.3 Éléments d'UML	69
C Guide d'installation de Novaskol	70
C.1 Installation locale (Desktop)	70
C.2 Configuration minimale requise	70
C.3 Installation sur hébergement web	70
D Structure du paquet de distribution	71
E Tableau des routes API	72

Table des figures

Liste des tableaux

1.1	Comparaison des frameworks PHP	20
2.1	Spécifications techniques de Novaskol	25
2.2	Modes de déploiement	28
2.3	Technologies backend	29
2.4	Technologies frontend	29
2.5	Technologies desktop	29
2.6	Technologies mobile	30
2.7	Liste des tables principales de la base de données	30
2.8	Dictionnaire de la table eleves	31
2.9	Dictionnaire de la table notes	32
2.10	Dictionnaire de la table bulletins	32
2.11	Modules d'administration générale	33
2.12	Modules de gestion des inscriptions	34
2.13	Modules de gestion pédagogique	34
2.14	Modules de communication	34
2.15	Modules RH	35
2.16	Modules financiers	35
2.17	Modules portail parents	36
2.18	Modules système	36
3.1	Besoins fonctionnels détaillés	37
3.2	Besoins non fonctionnels	38
3.3	Identification des acteurs	39
3.4	Correspondance acteurs-tâches	39
4.1	Outils de développement	51
4.2	Fichiers clés de l'application Electron	56
4.3	Comparaison Connecte Mobile vs Desktop	59
4.4	Modes de déploiement	60
E.1	Routes API de synchronisation	72

Introduction générale

L'informatique a profondément transformé notre société au cours des dernières décennies. La numérisation des processus administratifs et pédagogiques dans le domaine de l'éducation est devenue une nécessité incontournable pour améliorer l'efficacité, la transparence et la qualité du service rendu aux élèves, aux enseignants et aux parents.

Dans de nombreux établissements scolaires, en particulier dans les pays en développement et les zones rurales, la gestion des notes, des absences, des emplois du temps et de la communication avec les parents s'effectue encore de manière manuelle ou semi-automatisée à l'aide d'outils disparates. Cette situation engendre des problèmes majeurs : perte de temps, erreurs de saisie, difficultés de suivi, absence de centralisation des données, et communication limitée entre les différents acteurs de la communauté éducative.

Par ailleurs, les solutions de gestion scolaire existantes sur le marché présentent plusieurs limitations : elles sont souvent payantes, nécessitent une connexion Internet permanente, sont complexes à déployer, ou ne répondent pas aux spécificités du système éducatif local. La plupart des établissements scolaires ne disposent pas d'une infrastructure réseau fiable ni d'un accès Internet permanent, ce qui rend les solutions web traditionnelles inadaptées à leur contexte.

C'est dans ce contexte que s'inscrit notre projet de fin d'études : concevoir et réaliser une application de gestion scolaire complète, fonctionnant hors-ligne, déployable localement sur un simple PC sans nécessité de serveur dédié ni de connexion Internet. L'application, baptisée **Novaskol**, se présente sous la forme d'une application de bureau autonome (Electron + Laravel) couplée à une application mobile companion pour les usages mobiles, avec un mécanisme de synchronisation multi-appareils sur le réseau local.

Note personnelle. Mon parcours en Communication en Audiovisuelle et Numérique (CAN) m'a sensibilisé dès le début à la place du numérique dans notre société. Après la L3, j'ai naturellement orienté ma formation vers le développement web, fasciné par la capacité de créer des solutions accessibles, fonctionnelles et utiles. Bien que je ne vienne pas d'un cursus purement informatique, c'est précisément cette transition — du numérique vers la technique web — qui m'a poussé à choisir Novaskol comme projet. Je voyais dans ce système une opportunité de laisser une trace concrète, une solution que j'ai choisie, façonnée et portée de bout en bout, reflet de mon intérêt pour le web et de ma volonté de créer quelque chose qui a du sens.

Notre problématique centrale peut être formulée ainsi : **Comment concevoir et développer un système de gestion scolaire complet, accessible hors-ligne, facile à déployer et à utiliser, qui réponde aux besoins des établissements scolaires en matière de gestion administrative, pédagogique et financière ?**

Pour répondre à cette problématique, nous avons adopté une démarche méthodologique rigoureuse :

1. **Analyse des besoins** auprès des acteurs du système éducatif (direction, enseignants, parents, personnel administratif)
2. **Étude comparative** des solutions existantes et des technologies disponibles
3. **Conception architecturale** et modélisation UML complète
4. **Développement itératif** avec des cycles de test fréquents (méthodologie agile)
5. **Packaging et déploiement** pour une distribution facilitée via un installateur Windows

Les principales fonctionnalités offertes par Novaskol couvrent l'ensemble du cycle de gestion scolaire : inscription des élèves, gestion des classes et des matières, saisie et consultation des notes, génération de bulletins et de relevés, gestion des emplois du temps, suivi des présences par QR code, communication interne par messagerie instantanée, gestion financière (paiements, dépenses, salaires), gestion des ressources humaines, et synchronisation multi-appareils sur le réseau local.

Le système est articulé autour de 45 modules fonctionnels répartis en 8 catégories, 67 tables de base de données, et 33 migrations Laravel, le tout embarqué dans une application de bureau Electron de 650 Mo.

Ce mémoire est organisé en quatre chapitres :

Le premier chapitre présente l'état de l'art des technologies utilisées : architectures logicielles, modèle client/serveur, technologies web, frameworks PHP, systèmes de gestion de base de données, et applications de bureau hybrides.

Le deuxième chapitre est consacré à la présentation détaillée de Novaskol : son architecture globale, les technologies employées, sa base de données exhaustive, et l'ensemble de ses modules fonctionnels.

Le troisième chapitre détaille l'analyse et la conception du système à l'aide du langage UML : identification des acteurs, diagrammes de cas d'utilisation, de séquence, d'activité, et modèle conceptuel de données complet.

Le quatrième chapitre présente la réalisation pratique : environnement de développement, implémentation du backend et du frontend, application de bureau Electron avec son processus de démarrage, applications companion (mobile et desktop), processus de packaging et déploiement, et présentation des interfaces principales.

Enfin, une conclusion générale synthétise les résultats obtenus, les difficultés rencontrées, les compétences acquises, et propose des perspectives d'évolution pour le projet.

Chapitre 1

État de l’art et technologies utilisées

1.1 Introduction

La réalisation d’un système d’information moderne pour la gestion scolaire nécessite une maîtrise approfondie des concepts architecturaux et des technologies disponibles. Ce chapitre présente les fondements théoriques sur lesquels repose notre projet Novaskol : les architectures logicielles, le modèle client/serveur, les technologies web, les systèmes de gestion de base de données, et les frameworks modernes de développement d’applications de bureau et mobiles.

L’objectif de ce chapitre est de situer notre projet dans le paysage technologique actuel, de justifier nos choix techniques, et de fournir les bases conceptuelles nécessaires à la compréhension des chapitres suivants.

1.2 Architecture logicielle

1.2.1 Définition

L’architecture logicielle est la structure fondamentale d’un système logiciel, comprenant ses composants, leurs relations mutuelles et avec l’environnement, ainsi que les principes guidant sa conception et son évolution. Une bonne architecture permet de répondre aux exigences fonctionnelles et non fonctionnelles du système tout en facilitant sa maintenance et son évolution.

1.2.2 Le modèle 4+1 vues de Philippe Kruchten

Selon Philippe Kruchten (1995), l’architecture d’un système logiciel peut être représentée selon cinq vues complémentaires, chacune décrivant un aspect particulier du système :

Vue Logique : La vue logique se concentre sur la modélisation des principaux éléments d’architecture et mécanismes logiciels. Elle comprend les modèles d’analyse et de

conception du système, définissant les entités métier, leurs relations et leur comportement. Dans le cadre de Novaskol, cette vue correspond aux modèles métier : élèves, enseignants, classes, matières, notes, bulletins, etc.

Vue Composants : La vue composants identifie les modules logiciels qui implémentent les éléments définis dans la vue logique. Pour Novaskol, ces composants incluent les contrôleurs (30+), les services métier (9), les middlewares (3), les modèles Eloquent, et les modules fonctionnels de l'application.

Vue Processus : Cette vue définit les processus, la coordination et la synchronisation des tâches. Dans notre architecture, cela correspond au serveur PHP intégré, aux files d'attente (queues Laravel) pour les tâches asynchrones, à la synchronisation multi-appareils, et aux processus Electron (main process, render process).

Vue Déploiement : La vue déploiement précise l'architecture de production : ressources matérielles, implantation des composants, et configuration réseau. Novaskol peut être déployé selon trois modes : local (application de bureau autonome avec SQLite), hébergement (serveur web Apache/Nginx avec MySQL), ou hybride (desktop + appareils connectés).

Vue Use-Cases : Les cas d'utilisation constituent le fil conducteur qui relie les quatre autres vues. Ils expriment les besoins fonctionnels du système du point de vue des utilisateurs finaux.

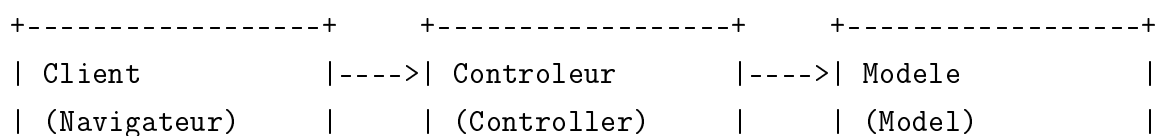
1.2.3 Le patron d'architecture MVC (Modèle-Vue-Contrôleur)

Le patron d'architecture MVC (Modèle-Vue-Contrôleur) est l'un des patrons d'architecture les plus utilisés dans le développement d'applications web. Il propose une séparation claire des responsabilités en trois composants distincts :

Le Modèle (Model) : Gère les données et la logique métier. Il encapsule l'accès à la base de données, les règles de validation, et les relations entre les entités. Dans Laravel, les modèles Eloquent ORM implémentent ce rôle, permettant d'interagir avec la base de données via une interface orientée objet.

La Vue (View) : Assure la présentation des données à l'utilisateur. Elle reçoit les données du contrôleur et les formate pour l'affichage. Novaskol utilise le moteur de templates Blade pour générer les interfaces HTML, avec TailwindCSS pour le style.

Le Contrôleur (Controller) : Fait le lien entre le modèle et la vue. Il traite les requêtes HTTP, orchestre la logique d'interaction, et prépare les données pour la vue.



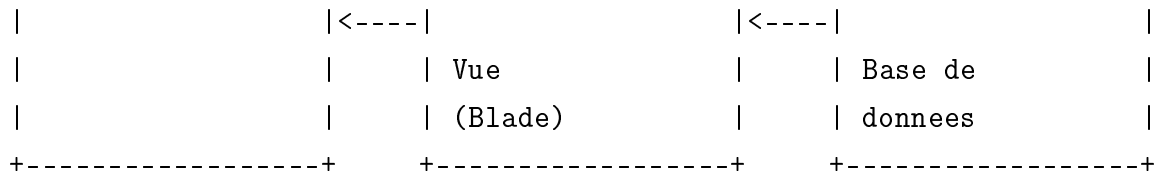


Figure I.1 : Schéma de l'architecture MVC (Modèle-Vue-Contrôleur)

1.3 Architectures distribuées

1.3.1 Définition

Une architecture distribuée est une architecture dont les ressources ne se trouvent pas au même endroit ou sur la même machine. Les fonctions de l'application sont réparties entre plusieurs systèmes interconnectés par un réseau.

1.3.2 Types d'architectures distribuées

Architecture centralisée : Toutes les ressources sont sur une seule machine. Le traitement, les données et la logique métier sont concentrés sur un seul système.

Architecture client/serveur : Répartition entre des clients (qui demandent des services) et un ou plusieurs serveurs (qui fournissent ces services).

Architecture peer-to-peer (P2P) : Chaque nœud est à la fois client et serveur.

Architecture multi-tiers (n-tiers) : L'application est divisée en plusieurs couches distinctes, chacune ayant une responsabilité spécifique.

1.4 Modèle Client/Serveur

1.4.1 Définition

Le modèle client/serveur est un modèle de communication où des machines clientes contactent un serveur pour obtenir des services. Un client envoie une requête au serveur, qui la traite et retourne une réponse.

1.4.2 Principe de fonctionnement

Le fonctionnement du modèle client/serveur suit un cycle requête-réponse en plusieurs étapes :

1. Le client établit une connexion avec le serveur
2. Le client envoie une requête décrivant l'opération souhaitée

3. Le serveur traite la requête (accès à la base de données, calculs, etc.)
4. Le serveur renvoie la réponse au client
5. Le client traite la réponse et l'affiche à l'utilisateur

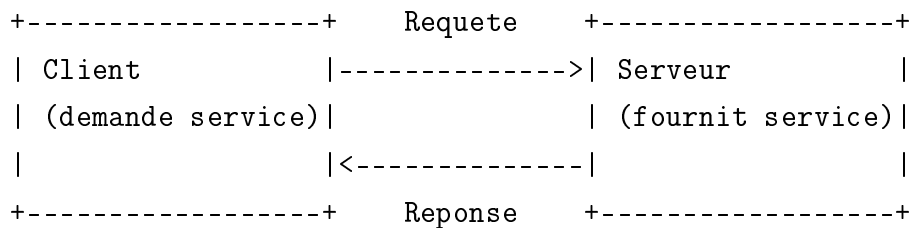


Figure I.2 : Principe de fonctionnement du Client/Serveur

1.4.3 Architecture à 2 niveaux (2-tiers)

Dans cette configuration, le client interagit directement avec le serveur de base de données.

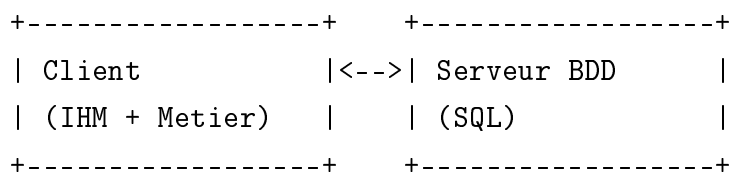


Figure I.3 : Architecture à 2 niveaux

1.4.4 Architecture à 3 niveaux (3-tiers)

L'architecture à trois niveaux introduit un niveau intermédiaire, le serveur d'application (middleware), qui contient la logique métier :

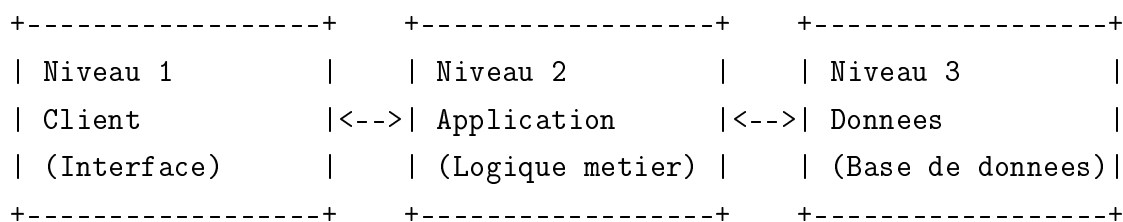


Figure I.4 : Architecture à 3 niveaux

Les avantages de cette architecture sont :

- Séparation claire des responsabilités
- Meilleure évolutivité (chaque niveau peut être mis à l'échelle indépendamment)
- Maintenance facilitée

- Sécurité renforcée

Application dans Novaskol :

- **Niveau 1 (Client)** : Navigateur web ou fenêtre Electron
- **Niveau 2 (Application)** : Serveur PHP/Laravel avec sa logique métier
- **Niveau 3 (Données)** : Base de données SQLite ou MySQL

1.4.5 Architecture multi-niveaux (n-tiers)

L'architecture multi-niveaux étend le modèle à 3 niveaux en décomposant davantage les couches. Novaskol, dans sa configuration Electron, utilise une variante de cette architecture avec les couches suivantes :

- Couche présentation : Interface utilisateur (HTML/Blade)
- Couche contrôle : Contrôleurs Laravel
- Couche service : Services métier
- Couche persistance : Modèles Eloquent
- Couche données : SQLite/MySQL

1.4.6 Middleware

Un middleware est un logiciel qui sert d'intermédiaire entre les différentes couches d'une architecture distribuée. Il facilite la communication, la gestion des transactions, et la coordination entre les composants.

Dans Laravel, les middlewares sont des filtres HTTP qui s'exécutent avant ou après le traitement d'une requête. Novaskol utilise un middleware personnalisé `module.access` pour le contrôle d'accès basé sur les modules.

1.5 Internet et le Web

1.5.1 Internet

Internet est un réseau mondial de réseaux interconnectés utilisant le protocole TCP/IP.

Principaux protocoles :

- **TCP/IP** : Protocole de base garantissant la transmission fiable des données
- **HTTP/HTTPS** : Protocole de transfert hypertexte
- **FTP** : Protocole de transfert de fichiers
- **SMTP/POP3/IMAP** : Protocoles de messagerie électronique
- **DNS** : Système de résolution de noms de domaine

1.5.2 Intranet et Extranet

- **Intranet** : Réseau interne à une organisation utilisant les technologies Internet
- **Extranet** : Extension de l'intranet permettant à des partenaires externes d'accéder à certaines ressources

1.5.3 Le World Wide Web

Sites web statiques

Composés de pages HTML pré-générées, servies telles quelles par le serveur web.

Sites web dynamiques

Les pages sont générées à la volée en fonction des paramètres de la requête et des données stockées en base. Ils utilisent des langages côté serveur (PHP, Python, Java, etc.).

1.5.4 Le protocole HTTP

HTTP (HyperText Transfer Protocol) est le protocole de communication utilisé par le Web. Novaskol utilise le serveur HTTP intégré de PHP (Artisan serve) pour le mode local, et peut être déployé sous Apache ou Nginx pour le mode hébergé.

1.5.5 REST API

REST (Representational State Transfer) est un style d'architecture pour les systèmes hypermédia distribués. Novaskol expose des API RESTful pour la synchronisation entre l'application desktop et les applications mobiles Connecte.

1.6 Technologies de développement web

1.6.1 PHP

Présentation

PHP (Hypertext Preprocessor) est un langage de programmation côté serveur, spécialement conçu pour le développement d'applications web. Créé par Rasmus Lerdorf en 1994, PHP est aujourd'hui l'un des langages les plus utilisés pour le développement web.

PHP 8.2

La version 8.2 de PHP, utilisée dans Novaskol, apporte des améliorations significatives :

- **Type system amélioré** : Types d'union, type mixed, types statiques
- **JIT (Just-In-Time Compilation)** : Compilation à la volée pour des performances accrues
- **Named arguments** : Arguments nommés pour les fonctions
- **Match expression** : Alternative améliorée au switch
- **Readonly classes** : Classes immutables
- **Performance** : Jusqu'à $3\times$ plus rapide que PHP 7.x

Avantages de PHP pour Novaskol

- Large communauté et écosystème riche
- Support natif de SQLite (extension PDO_SQLITE)
- Facilité d'apprentissage et de déploiement
- Compatibilité avec tous les hébergeurs
- Performances accrues avec PHP 8.x
- Bibliothèque standard riche

1.6.2 Laravel

Présentation

Laravel est un framework PHP open-source, créé par Taylor Otwell en 2011, qui suit le patron d'architecture MVC. Il est aujourd'hui le framework PHP le plus populaire.

Version utilisée : Laravel 12

Laravel 12, utilisé dans Novaskol, offre les fonctionnalités suivantes :

Eloquent ORM : Mappage objet-relationnel intuitif.

Blade Template Engine : Moteur de templates performant avec héritage de vues, sections, composants.

Migration System : Versionnement du schéma de base de données.

Artisan CLI : Outil en ligne de commande pour les tâches courantes.

Queues : Gestion de tâches asynchrones.

Security : Protection intégrée contre les vulnérabilités web courantes.

Testing : PHPUnit intégré avec des helpers de test.

Fonctionnalités clés utilisées dans Novaskol

- Service Providers
- Middleware (module.access)
- Event & Listeners
- Filesystem
- Cache
- Validation
- Mail (notifications)
- Routing

1.6.3 Comparaison des frameworks PHP

TABLE 1.1 – Comparaison des frameworks PHP

Critère	Laravel	Symfony	CodeIgniter	CakePHP
Courbe d'apprentissage	Facile	Modérée	Facile	Modérée
Performance	Bonne	Bonne	Très bonne	Bonne
ORM	Eloquent	Doctrine	Custom	CakePHP ORM
Templating	Blade	Twig	Custom	Custom
Communauté	Très large	Large	Modérée	Modérée
Version actuelle	12.x	7.x	4.x	5.x

1.6.4 Node.js et npm

Node.js est un environnement d'exécution JavaScript côté serveur. npm est le gestionnaire de paquets associé.

Dans Novaskol, ils sont utilisés pour :

- Le processus principal Electron
- Les outils de build (Vite, TailwindCSS, electron-builder)
- La gestion des dépendances frontend
- Les scripts de développement et de build

1.6.5 Vite

Vite est un outil de build moderne qui offre un développement rapide avec rechargement à chaud (HMR). Novaskol utilise Vite 7 pour compiler les assets CSS (TailwindCSS) et JavaScript.

1.6.6 TailwindCSS

TailwindCSS est un framework CSS utilitaire avec une approche "utility-first". Version 4 utilisée dans Novaskol, avec configuration automatique.

1.7 Systèmes de Gestion de Base de Données (SGBD)

1.7.1 Définition

Un Système de Gestion de Base de Données (SGBD) est un logiciel permettant de stocker, manipuler et interroger des données de manière structurée. Il assure la persistance, l'intégrité, la sécurité et la gestion des accès concurrents.

1.7.2 SQLite

SQLite est un SGBD relationnel embarqué, léger et sans serveur, stockant les données dans un fichier unique.

Caractéristiques :

- Aucune configuration serveur nécessaire
- Base de données = fichier unique
- Performant pour applications mono-utilisateur
- Supporté nativement par PHP
- Taille maximale : 281 To

Utilisation dans Novaskol : Base de données par défaut pour le mode hors-ligne.

1.7.3 MySQL/MariaDB

MySQL est un SGBD relationnel client/serveur. MariaDB est un fork communautaire.

Caractéristiques :

- Architecture client/serveur avec processus dédié
- Gestion multi-utilisateurs avancée
- Performances élevées
- Transactions ACID
- Procédures stockées, triggers, vues

Utilisation dans Novaskol : Déploiements hébergés et installations multi-utilisateurs.

1.8 Applications de bureau hybrides

1.8.1 Electron

Electron est un framework open-source (GitHub, 2013) permettant de créer des applications de bureau multiplateformes avec HTML/CSS/JavaScript. Utilise Chromium pour le rendu et Node.js pour les API système.

Version utilisée : Electron 37.2.1

Architecture :

- **Main process (Node.js)** : Cycle de vie, fenêtres, API système, IPC
- **Renderer process (Chromium)** : Interface web, accès limité système

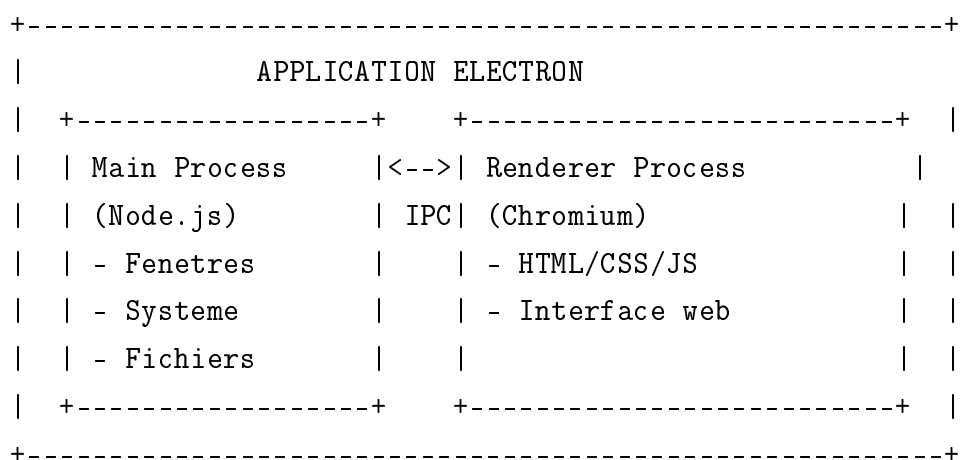


Figure I.5 : Architecture d'une application Electron

Avantages pour Novaskol :

- Packaging complet PHP + Laravel
- Déploiement simplifié (installateur unique)
- Fonctionnement hors-ligne
- Accès API système
- Expérience native

1.8.2 electron-builder

Outil de packaging pour applications Electron. Version 26.0.12 utilisée avec cible NSIS Windows.

Configuration :

- Installateur non one-click avec choix du répertoire
- Raccourcis Bureau et Menu Démarrer
- Ressources Laravel intégrées

1.8.3 Capacitor

Capacitor est un framework de développement d'applications mobiles hybrides, développé par l'équipe Ionic.

Version utilisée : Capacitor 8.3.3

Avantages :

- Accès aux API natives (appareil photo, notifications, etc.)
- Génération de builds Android et iOS à partir d'une base de code web
- Performance proche du natif

1.9 Conclusion

Ce chapitre nous a permis de poser les bases théoriques et technologiques de notre projet. Le modèle client/serveur à 3 niveaux, couplé à l'architecture MVC de Laravel, constitue le socle architectural de Novaskol. Le choix des technologies (PHP 8.2, Laravel 12, SQLite, Electron 37, Capacitor 8) a été guidé par les impératifs de simplicité de déploiement, de fonctionnement hors-ligne, de maintenabilité et d'évolutivité.

Chapitre 2

Architecture et technologies de Novaskol

2.1 Introduction

Ce chapitre est consacré à la présentation détaillée de Novaskol : son contexte, son architecture globale, ses composants technologiques, sa base de données exhaustive, et l'ensemble des modules fonctionnels qui le composent.

2.2 Présentation du projet Novaskol

2.2.1 Contexte et objectifs

Novaskol est un système de gestion scolaire complet conçu pour répondre aux besoins des établissements d'enseignement primaire, secondaire et technique. Le projet est né du constat que de nombreuses écoles, en particulier dans les zones à connectivité Internet limitée, ne disposent pas d'outils numériques adaptés à leur contexte.

L'auteur du projet est **Tojo Nambinina RANDRIAMIFALY**, et le projet est publié sous licence MIT.

Objectifs principaux :

1. Fournir un outil de gestion scolaire complet et accessible hors-ligne
2. Simplifier le déploiement via une application de bureau autonome
3. Permettre une synchronisation multi-appareils sur le réseau local
4. Offrir une interface adaptée à chaque profil d'utilisateur
5. Garantir la sécurité et l'intégrité des données
6. Assurer la portabilité sur Windows 10/11

TABLE 2.1 – Spécifications techniques de Novaskol

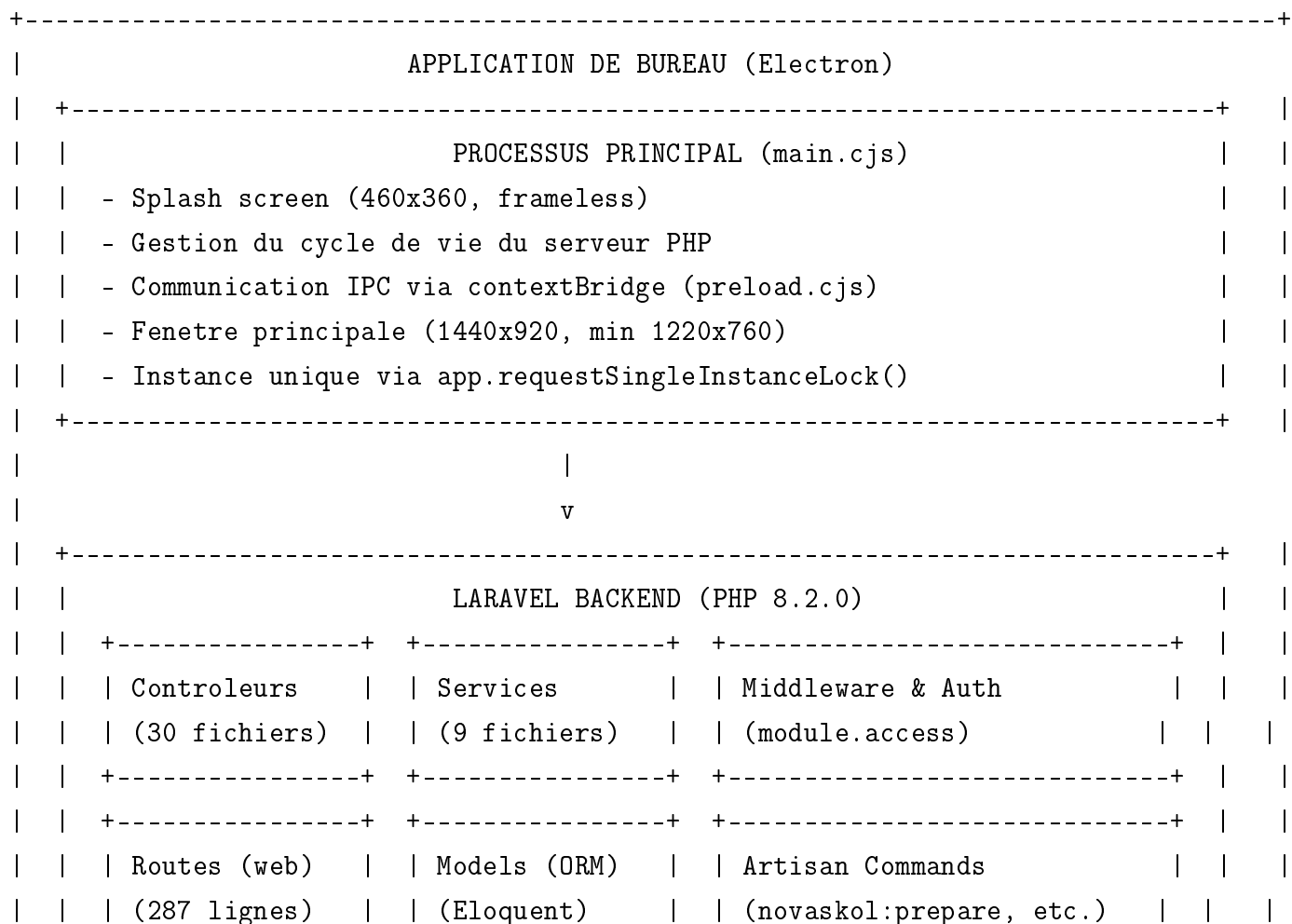
Spécification	Valeur
Version	1.0.6
Backend	Laravel 12.x (PHP 8.2+)
Application de bureau	Electron 37.2.1
Base de données	SQLite (défaut) / MySQL 8.0 / MariaDB
Application mobile	React 19 + Capacitor 8 (Android + PWA)
Companion Desktop	Electron 37 (Windows)
Licence	MIT
Auteur	Tojo Nambinina RANDRIAMIFALY

2.2.2 Spécifications techniques

2.3 Architecture globale du système

2.3.1 Vue d'ensemble

L'architecture de Novaskol est organisée en plusieurs couches interconnectées, formant un écosystème complet de gestion scolaire :



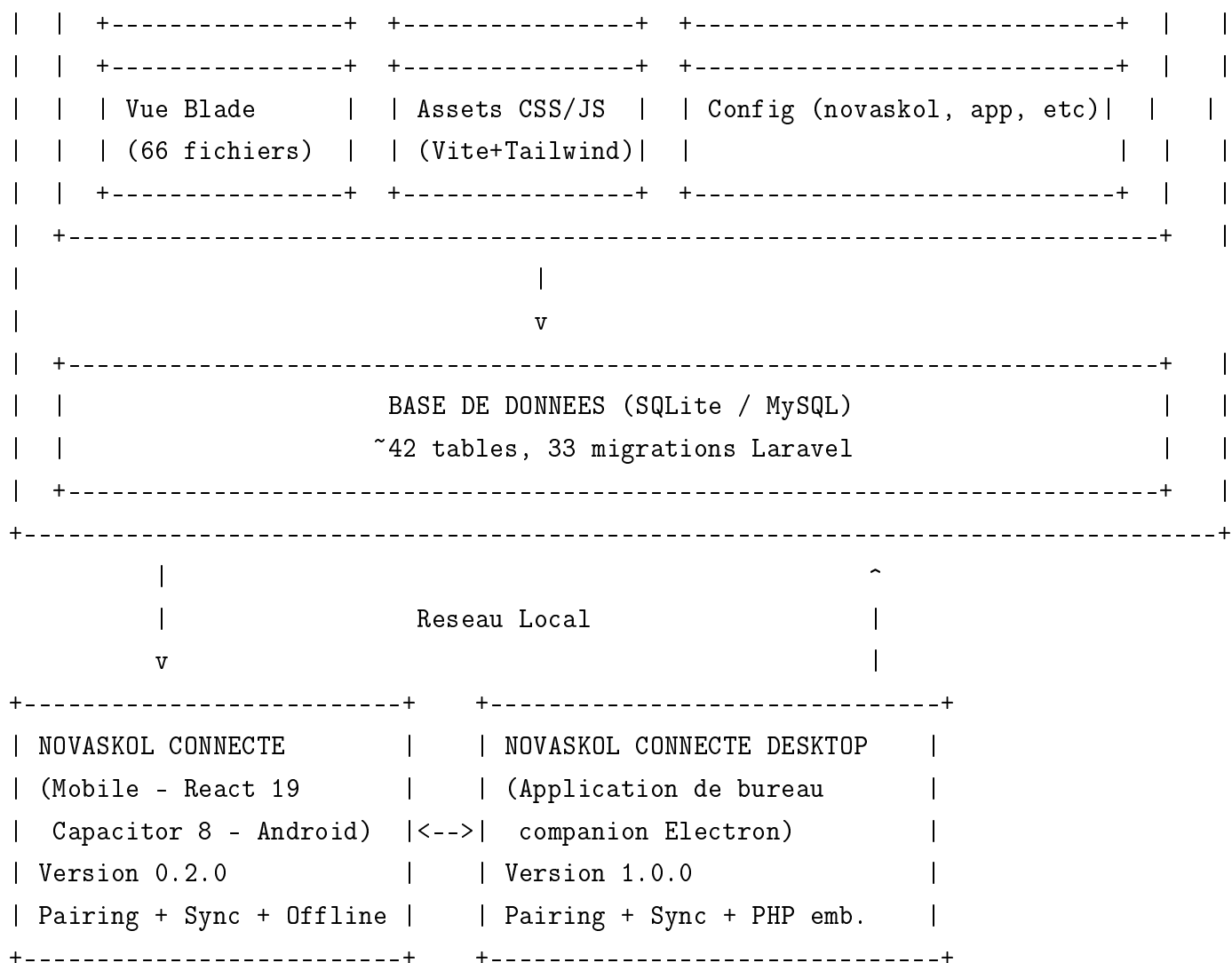


Figure II.1 : Architecture globale de Novaskol

2.3.2 L'application de bureau Electron (Novaskol Desktop)

Composants embarqués

- **Serveur PHP portable** : PHP 8.2.0 avec 19 extensions activées
- **Serveur MySQL/MariaDB portable** : MySQL 8.0.31
- **Application Laravel complète** avec toutes ses dépendances PHP
- **Base de données SQLite** par défaut, sans configuration

Processus de démarrage

Le processus de démarrage de l'application suit une séquence précise :

1. L'utilisateur lance `Novaskol.exe`
2. Une fenêtre de démarrage (splash screen) s'affiche

3. Le processus principal Electron localise le binaire PHP
4. Le fichier `.env` est configuré automatiquement
5. La clé d'application (`APP_KEY`) est générée si absente
6. Les migrations de base de données sont exécutées
7. Le serveur PHP intégré est démarré
8. L'application attend que le serveur HTTP réponde
9. La fenêtre principale charge l'URL

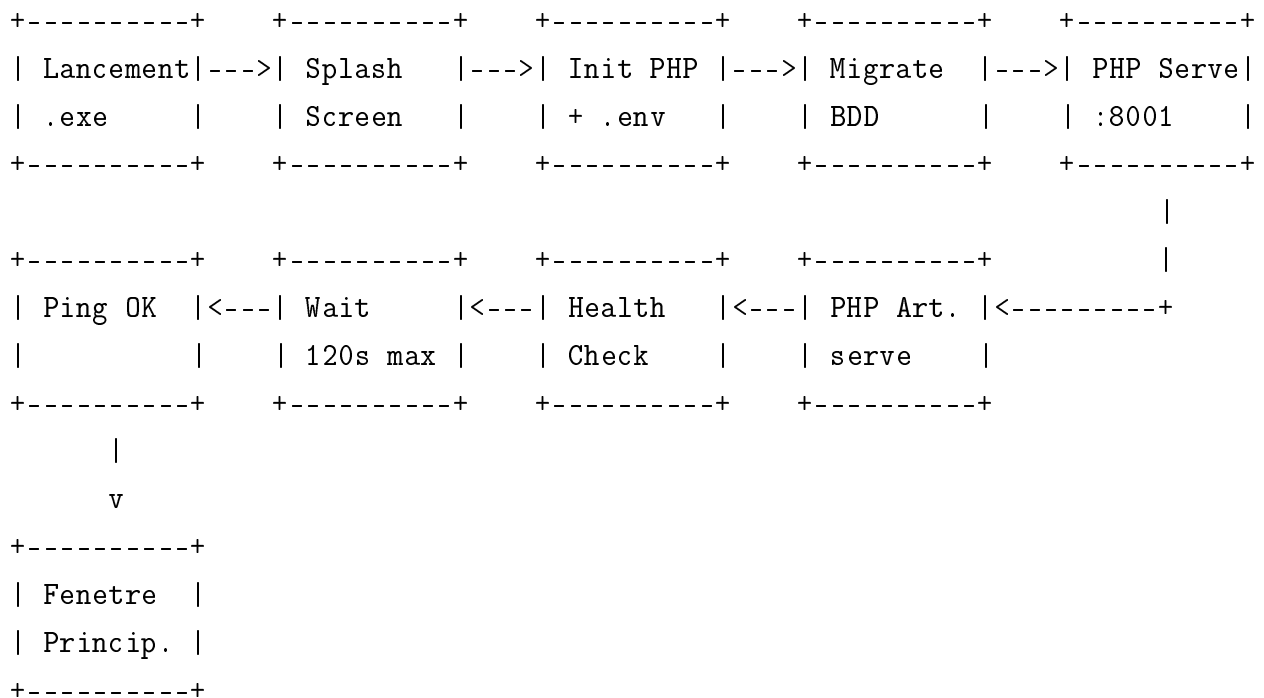


Figure II.2 : Processus de démarrage de l'application Electron

Communication IPC

La communication entre le processus principal (Node.js) et le processus de rendu (Chromium) s'effectue via un pont IPC sécurisé utilisant `contextBridge` :

```
// preload.cjs - Pont IPC securise
contextBridge.exposeInMainWorld('desktopShell', {
  onSplashStatus: (cb) => ipcRenderer.on('splash-status', (_, m)
    => cb(m)),
  getMeta: () => ipcRenderer.invoke('desktop:get-meta'),
  print: () => ipcRenderer.send('print'),
  printHtml: (html) => ipcRenderer.send('print-html', html)
});
```

Gestion du cycle de vie

- **Démarrage** : Splash screen → Initialisation → Fenêtre principale
- **Fermeture** : Boîte de confirmation → Arrêt du serveur PHP → Nettoyage
- **Instance unique** : Lock d'instance pour éviter les lancements multiples

2.3.3 La synchronisation multi-appareils

Processus d'appairage

1. L'appareil secondaire scanne un QR code ou saisit un code d'appairage
2. Il ping le serveur principal via `/reseau-local/manifest-appareil`
3. Il envoie ses informations (UUID, type, plateforme, etc.)
4. Le serveur vérifie le code et autorise la connexion
5. Les données initiales sont transférées (bootstrap)

Synchronisation bidirectionnelle

- **Push** : L'appareil connecté envoie ses modifications locales au serveur principal
- **Pull** : L'appareil connecté récupère les modifications des autres appareils
- **Intervalle** : Synchronisation automatique toutes les 5 minutes
- **File d'attente** : Les modifications sont mises en file d'attente en cas de déconnexion

Gestion des conflits

1. Les deux versions sont conservées (locale et entrante)
2. Un ticket de conflit est créé dans la table `sync_conflicts`
3. L'administrateur peut résoudre le conflit manuellement
4. La résolution choisie est propagée aux autres appareils

2.3.4 Modes de déploiement

TABLE 2.2 – Modes de déploiement

Mode	Base de données	Usage
Local (Desktop)	SQLite	École individuelle, hors-ligne
Hébergement	MySQL/MariaDB	Accès web permanent
Hybride	SQLite + Sync réseau	Multi-poste local

2.4 Technologies et outils utilisés

2.4.1 Backend

TABLE 2.3 – Technologies backend

Technologie	Rôle
PHP 8.2.0	Langage serveur
Laravel 12	Framework MVC
Composer 2.x	Gestionnaire de dépendances PHP
barryvdh/laravel-dompdf ^{2.3}	Génération de PDF
phpoffice/phpspreadsheet ^{5.7}	Manipulation de fichiers Excel
chillerlan/php-qrcode ^{5.0}	Génération de QR codes
laravel/pint	Correcteur de style

2.4.2 Frontend

TABLE 2.4 – Technologies frontend

Technologie	Rôle
Blade	Moteur de templates Laravel
TailwindCSS 4	Framework CSS utilitaire
Vite 7	Outil de build
JavaScript (vanilla + jQuery 3.6)	Interactivité client
Chart.js 4	Graphiques statistiques
FullCalendar 6	Calendrier interactif
Font Awesome 6	Icônes
SweetAlert2 11.x	Boîtes de dialogue
Axios	Requêtes HTTP

2.4.3 Desktop

TABLE 2.5 – Technologies desktop

Technologie	Rôle
Electron 37.2.1	Framework d'application de bureau
electron-builder 26.0.12	Packaging NSIS
Node.js 22	Environnement d'exécution

TABLE 2.6 – Technologies mobile

Technologie	Rôle
React 19	Framework d'interface utilisateur
Capacitor 8.3.3	Pont natif pour applications mobiles
Vite 8	Outil de build
Service Worker	Cache hors-ligne PWA

2.4.4 Mobile

2.5 Architecture détaillée de la base de données

2.5.1 Présentation

La base de données de Novaskol comprend environ 42 tables organisées en plusieurs domaines fonctionnels. Le système de migration de Laravel (33 migrations) permet de versionner et de déployer le schéma automatiquement.

2.5.2 Liste des tables principales

TABLE 2.7: Liste des tables principales de la base de données

#	Table	Objectif
1	utilisateurs	Utilisateurs de l'application
2	ecole	Informations sur l'établissement
3	parametres	Paramètres de configuration
4	permissions	Permissions d'accès aux modules
5	classes	Classes (nom, niveau)
6	eleves	Élèves inscrits
7	matieres	Matières enseignées
8	classe_matieres	Association classe-matière (coefficient)
9	professeurs	Enseignants
10	professeurs_classes	Affectation enseignant-classe-matière
11	notes	Notes des élèves
12	bulletins	Bulletins de notes
13	emploi_du_temps	Emploi du temps
14	evenements	Événements du calendrier scolaire
15	presence_eleves	Présence des élèves

Suite de la liste des tables

#	Table	Objectif
16	parents	Parents/tuteurs
17	parent_eleves	Lien parent-élève
18	enseignants	Table enseignants alternative
19	staff	Personnel non enseignant
20	departements	Départements
21	conversations	Conversations
22	conversation_participants	Participants aux conversations
23	messages	Messages
24	notifications	Notifications système
25	types_paiements	Types de paiement
26	paiements	Paiements effectués
27	depenses	Dépenses
28	salaires_assignes	Salaires du personnel
29	roles	Rôles utilisateur
30	dossiers	Dossiers documentaires
31	fichiers	Fichiers dans les dossiers
32	equipements	Équipements / inventaire
33	salles	Salles
34	reservations	Réservations de salles
35	licence	Licence logicielle
36	sync_devices	Appareils connectés
37	sync_batches	Lots de synchronisation
38	sync_changes	Modifications synchronisées
39	sync_conflicts	Conflits de synchronisation
40	sync_record_keys	Clés UUID synchronisées
41	teacher_lessons	Plans de cours des enseignants
42	teacher_tasks	Tâches des enseignants

2.5.3 Dictionnaire des données

Table eleves :

TABLE 2.8: Dictionnaire de la table eleves

Champ	Type	Description
id	BIGINT PK	Identifiant unique

Suite de la table eleves

Champ	Type	Description
prenom	VARCHAR(100)	Prénom de l'élève
nom	VARCHAR(100)	Nom de l'élève
date_naissance	DATE	Date de naissance
lieu_naissance	VARCHAR(150)	Lieu de naissance
adresse	VARCHAR(255)	Adresse de l'élève
numero_acte	VARCHAR(100)	Numéro d'acte de naissance
id_classe	BIGINT FK	Classe de l'élève
matricule	VARCHAR(100)	Matricule unique
annee_scolaire	VARCHAR(20)	Année scolaire
genre	ENUM('F','G')	Genre
statut	ENUM('nouveau','passant','redoublant')	Statut
photo	VARCHAR(255)	Photo d'identité
qr_token	VARCHAR(100)	Token QR pour carte

Table notes :

TABLE 2.9 – Dictionnaire de la table notes

Champ	Type	Description
id	BIGINT PK	Identifiant unique
eleve_id	BIGINT FK	Élève concerné
matiere_id	BIGINT FK	Matière
professeur_id	BIGINT FK	Professeur
valeur	FLOAT	Note (0-20)
trimestre	INTEGER	Trimestre (1-3)
coefficient	FLOAT	Coefficient
periode	VARCHAR(10)	Période

Table bulletins :

TABLE 2.10 – Dictionnaire de la table bulletins

Champ	Type	Description
id	BIGINT PK	Identifiant unique
id_eleve	BIGINT FK	Élève
trimestre	INTEGER	Trimestre
moyenne	FLOAT	Moyenne générale
mention	VARCHAR(100)	Mention obtenue
appreciation	TEXT	Appréciation
annee_scolaire	VARCHAR(20)	Année scolaire

2.5.4 Relations principales

- Un **Élève** appartient à une **Classe** (N :1 via `id_classe`)
- Une **Classe** a plusieurs **Matières** (N :N via `classe_matières`)
- Un **Professeur** enseigne dans plusieurs **Classes** et **Matières** (N :N via `professeurs_classes`)
- Un **Élève** a plusieurs **Notes** par matière (1 :N)
- Un **Bulletin** est lié à un **Élève** et un trimestre (N :1)
- Un **Parent** est lié à plusieurs **Élèves** (N :N via `parent_eleves`)
- Un **Message** appartient à une **Conversation** (N :1)
- Un **Paiement** est lié à un **Type de paiement** (N :1)

2.5.5 Système de synchronisation

Les tables de synchronisation forment un système robuste de réplication de données :

- `sync_devices` : Enregistre chaque appareil connecté
- `sync_batches` : Regroupe les modifications par lot
- `sync_changes` : Stocke chaque modification
- `sync_conflicts` : Enregistre les conflits
- `sync_record_keys` : Maintient la correspondance IDs locaux / UUIDs globaux

2.6 Modules et fonctionnalités

Novaskol propose 45 modules fonctionnels répartis en 8 catégories.

2.6.1 Administration générale

TABLE 2.11 – Modules d’administration générale

Module	Route	Description
Dashboard / Accueil	/dashboard	Tableau de bord avec statistiques
École	/ecole	Gestion des informations de l’établissement
Paramètres	/parametres	Configuration système, sauvegardes
Comptes utilisateurs	/comptes-utilisateurs	Gestion des comptes

TABLE 2.12 – Modules de gestion des inscriptions

Module	Route	Description
Inscription	/inscription	Enregistrement des élèves, import Excel
Liste des classes	/classes	CRUD des classes
Matières	/matieres	Gestion des matières, coefficients
Dépôt de dossier	/depot-dossier	Gestion des documents administratifs

TABLE 2.13 – Modules de gestion pédagogique

Module	Route	Description
Notes	/notes	Saisie des notes par matière et trimestre
Bulletins	/bulletin	Génération de bulletins
Résultats	/resultats	Consultation des résultats
Examens blancs	/examen-blanc	Gestion des examens blancs
Emploi du temps	/emploi-du-temps	Planification des cours
Présence élèves	/presence-etudiant	Suivi des présences par QR code
Calendrier	/calendrier	Calendrier des événements scolaires
Carte scolaire	/cartes	Génération de cartes avec QR code
Espace enseignant	/enseignant/espace	Plans de cours, progressions

2.6.2 Gestion des inscriptions

2.6.3 Gestion pédagogique

2.6.4 Communication

TABLE 2.14 – Modules de communication

Module	Route	Description
Messagerie générale	/communication	Chat général
Messagerie privée	/chat-prive	Chat privé 1-1
Messagerie groupe	/chat-groupe	Chat de groupe
Notifications	/notifications	Notifications système

TABLE 2.15 – Modules RH

Module	Route	Description
Enseignants	/enseignants	Gestion des enseignants
Personnels	/staff	Gestion du personnel non enseignant
Présence personnels	/presence	Suivi des présences enseignants
Permissions	/permissions	Contrôle d'accès aux modules
Gestion ressources	/gestion-ressource	Salles, équipements, ressources

TABLE 2.16 – Modules financiers

Module	Route	Description
Types de paiement	/detail-paiement/type	Configuration frais de scolarité
Comptabilité	/comptable	Enregistrement des paiements
Liste des paiements	/liste-paiements	Consultation des transactions
Factures	/facture	Génération de factures et reçus
Rapports comptables	/rapport-comptable	États financiers
Salaires	/detail-paiement/salaires	Gestion des salaires

2.6.5 Gestion des ressources humaines

2.6.6 Gestion financière

2.6.7 Portail parents

2.6.8 Système

2.7 Conclusion

Ce chapitre a présenté l'architecture globale et les composants de Novaskol. Le système combine un backend Laravel robuste, une application de bureau Electron autonome embarquant PHP et SQLite, et deux applications companion. Avec 45 modules fonctionnels couvrant 8 domaines et une base de données de 42 tables, Novaskol offre une couverture complète des besoins de gestion d'un établissement scolaire.

TABLE 2.17 – Modules portail parents

Module	Route	Description
Espace parent	/parent/espace	Suivi résultats, présence, paiements
Communication parent	/parent/responsable-chat	Chat avec les enseignants

TABLE 2.18 – Modules système

Module	Route	Description
Diagnostics	/diagnostic-systeme	Vérification état du système
Sauvegardes	/sauvegardes	Création, téléchargement, restauration
Réseau local	/reseau-local	Appairage, synchronisation
Guide d'utilisation	/guide-utilisation	Aide intégrée
À propos	/apropos-novaskol	Informations version

Chapitre 3

Analyse et conception

3.1 Introduction

Ce chapitre est consacré à l'analyse et à la conception du système Novaskol à l'aide du langage de modélisation UML (Unified Modeling Language). Nous présentons successivement l'analyse des besoins, l'identification des acteurs, les diagrammes de cas d'utilisation, les diagrammes de séquence, les diagrammes d'activité, et le modèle conceptuel de données.

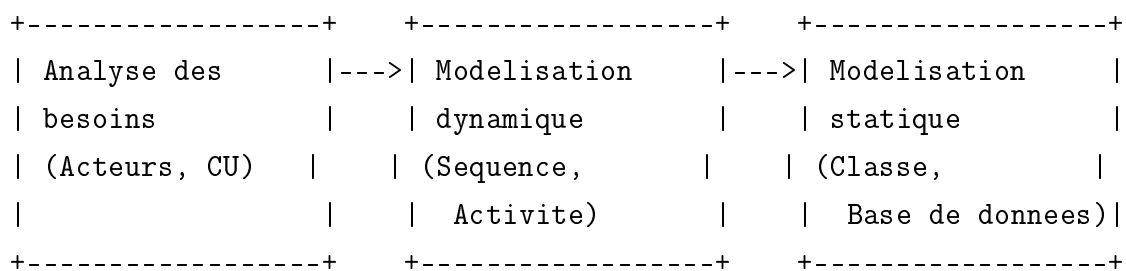


Figure III.1 : Démarche de modélisation de l'application

3.2 Analyse des besoins

3.2.1 Besoins fonctionnels

TABLE 3.1: Besoins fonctionnels détaillés

ID	Besoin	Module concerné
BF01	Authentification	Auth
BF02	Gestion des profils	Comptes
BF03	Gestion des élèves	Inscription

Suite des besoins fonctionnels		
ID	Besoin	Module concerné
BF04	Gestion des classes	Classes
BF05	Gestion des matières	Matières
BF06	Saisie des notes	Notes
BF07	Génération de bulletins	Bulletin
BF08	Emploi du temps	Emploi du temps
BF09	Suivi des présences	Présence
BF10	Communication	Communication
BF11	Gestion financière	Comptable
BF12	Gestion RH	RH
BF13	Calendrier	Calendrier
BF14	Sauvegardes	Sauvegardes
BF15	Synchronisation	Réseau local
BF16	Rapports	Rapports

3.2.2 Besoins non fonctionnels

TABLE 3.2 – Besoins non fonctionnels		
ID	Besoin	Priorité
BNF01	Hors-ligne	Haute
BNF02	Performance ($< 2s$)	Haute
BNF03	Sécurité (bcrypt, accès contrôlés)	Haute
BNF04	Portabilité Windows 10/11	Haute
BNF05	Évolutivité	Moyenne
BNF06	Maintenabilité (MVC, PSR-4)	Moyenne
BNF07	Sauvegarde automatique	Haute
BNF08	Interface intuitive	Haute
BNF09	Multilingue (6 langues)	Basse
BNF10	Sécurité des accès	Haute

TABLE 3.3 – Identification des acteurs

Acteur	Description	Cas d'utilisation principaux
Administrateur	Gère l'ensemble du système	Gestion complète
Enseignant	Saisit les notes, gère les présences	Saisie notes, présence
Élève	Consulte ses notes, emploi du temps	Consultation
Parent	Consulte les résultats de ses enfants	Consultation, communication
Agent de scolarité	Gère les inscriptions, paiements	Inscription, paiements
Chef d'établissement	Supervise, consulte les rapports	Rapports, supervision
Personnel (staff)	Gère les ressources, logistique	RH, ressources

TABLE 3.4 – Correspondance acteurs-tâches

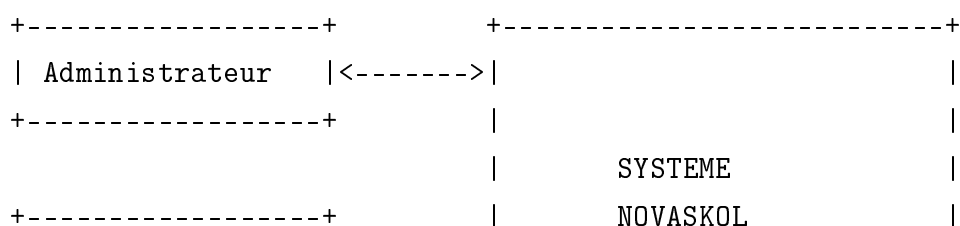
Tâche	Admin	Ens.	Élève	Parent	Scol.	Chef	Staff
Authentification	X	X	X	X	X	X	X
Gestion élèves	X				X	X	
Saisie notes		X				X	
Consultation notes	X	X	X	X	X	X	X
Bulletins	X	X		X	X	X	
Emploi du temps	X	X	X	X	X	X	
Présence élèves	X	X		X		X	
Communication	X	X	X	X	X	X	X
Paievements	X			X	X	X	
Gestion RH	X					X	
Rapports	X				X	X	
Paramètres	X						
Synchronisation	X						

3.3 Identification des acteurs

3.3.1 Correspondance acteurs-tâches

3.4 Diagrammes de cas d'utilisation

3.4.1 Diagramme de contexte



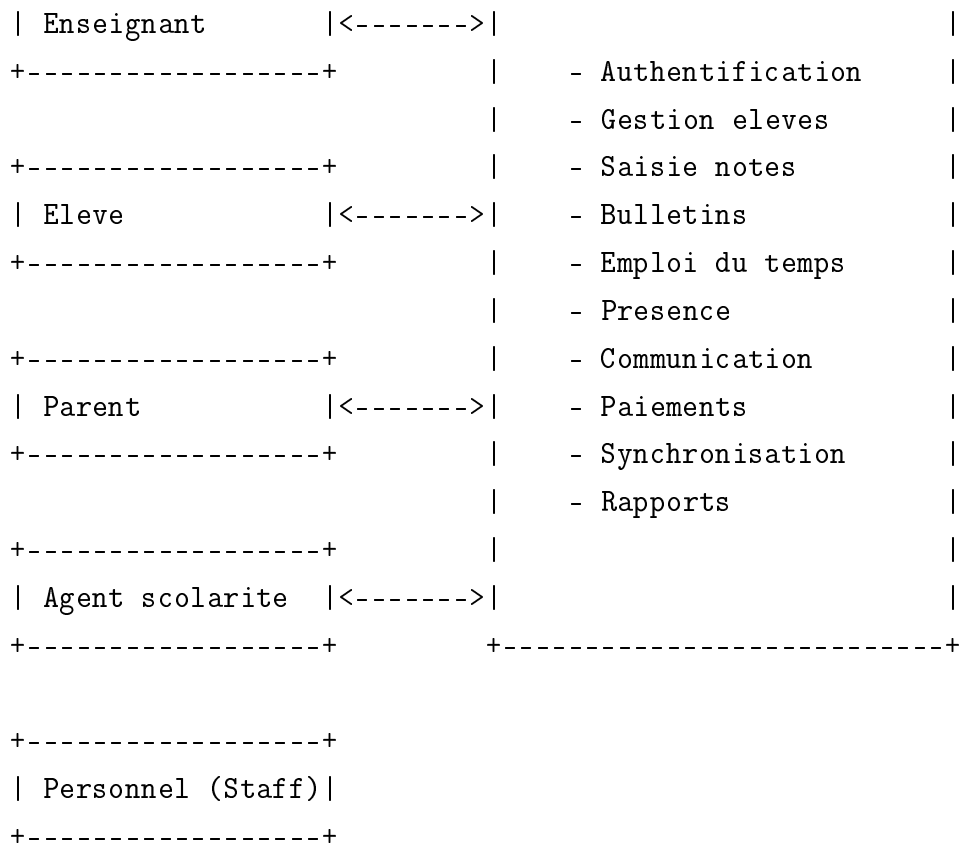


Figure III.2 : Diagramme de contexte

3.4.2 Cas d'utilisation par acteur

Administrateur :

- Gérer l'établissement
- Gérer les utilisateurs
- Gérer la base de données (sauvegardes, restauration)
- Configurer les paramètres système
- Accéder au diagnostic système
- Gérer la synchronisation réseau
- Consulter les rapports

Enseignant :

- S'authentifier
- Consulter son emploi du temps
- Saisir les notes des élèves
- Gérer les présences des élèves
- Planifier les cours


```

| |      +--- <<include>> --> Charger | | | |
| |      |               liste eleves| |
| |      |               |           | |
| |      +--- <<include>> --> Valider | |
| |      |               notes      | |
| |      |               |           | |
| |      +--- <<include>> --> Calculer | |
| |      |               moyennes   | |
| +-----+
| +-----+
| | Selectionner classe + matiere | |
| +-----+
+-----+

```

Figure III.4 : Diagramme de cas d'utilisation – Saisie des notes

3.5 Diagrammes de séquence

3.5.1 Diagramme de séquence – Authentification

Utilisateur	Controleur	Base de donnees	Session
-- email, pwd, role -->			
	-- SELECT ----->		
	<-- user data -----		
	-- Verifier hash --		
	(si valide)		
	-- Creer session ----->		
	-- Charger perms ->		
	<-- permissions ---		
<-- Redirection dashboard -----			
	(si invalide)		
<-- Message d'erreur -----			

Figure III.5 : Diagramme de séquence – Authentification

3.5.2 Diagramme de séquence – Saisie des notes

Enseignant	Controleur	BDD (notes)	BDD (eleves)
------------	------------	-------------	--------------

4. Calcul du rang (classement des élèves par moyenne générale)
5. Détermination de la mention en fonction de la moyenne
6. Génération du bulletin en HTML, puis export PDF via DOMPDF

3.5.4 Diagramme de séquence – Synchronisation

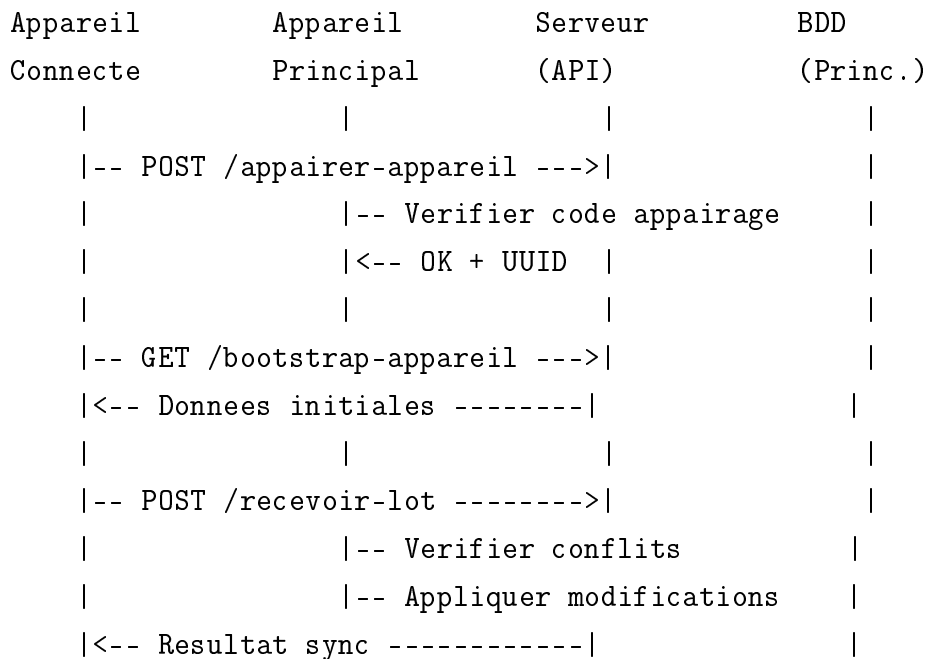
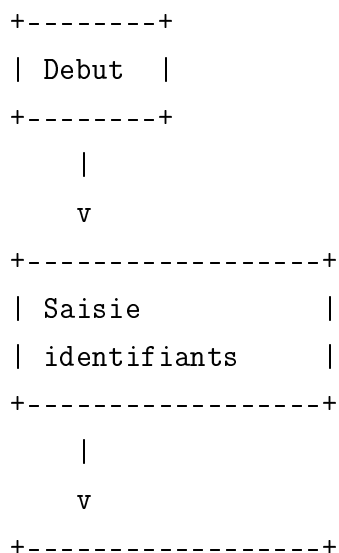


Figure III.8 : Diagramme de séquence – Synchronisation

3.6 Diagrammes d'activité

3.6.1 Diagramme d'activité – Authentification



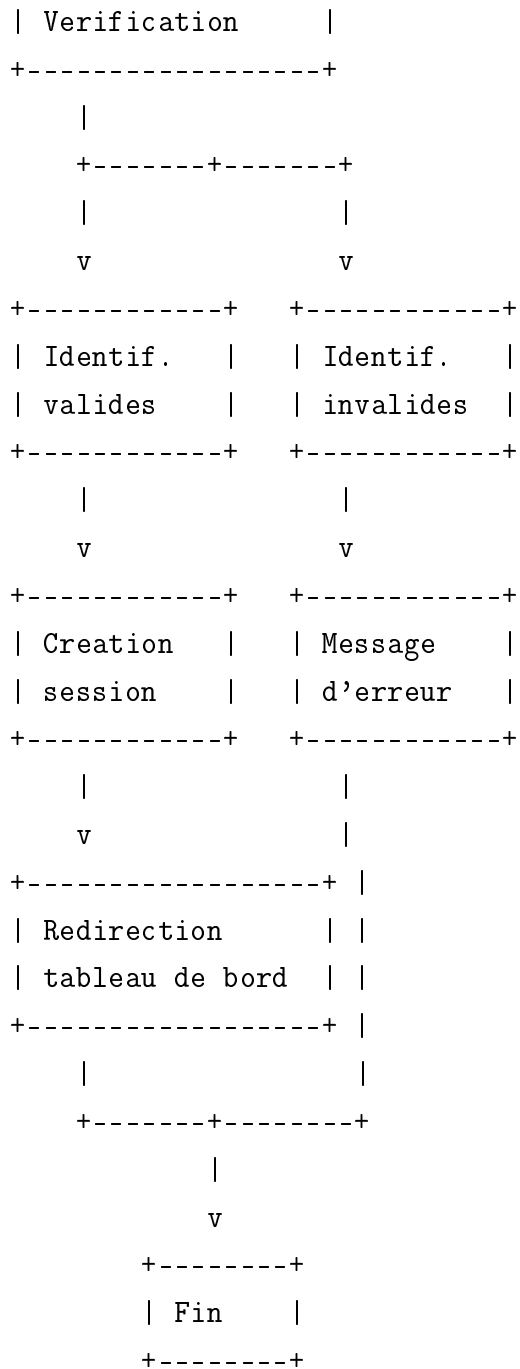
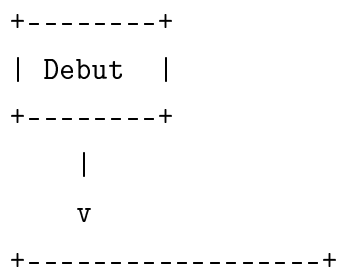
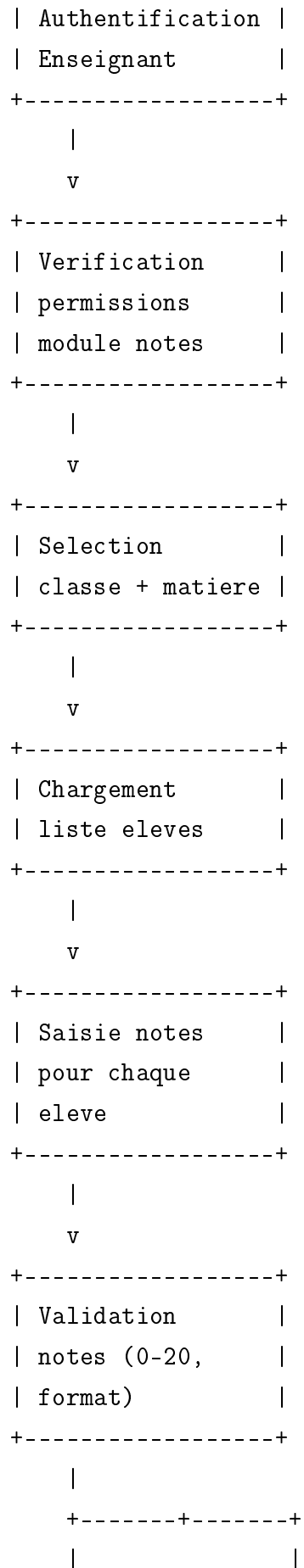


Figure III.9 : Diagramme d'activité – Authentification

3.6.2 Diagramme d'activité – Saisie des notes





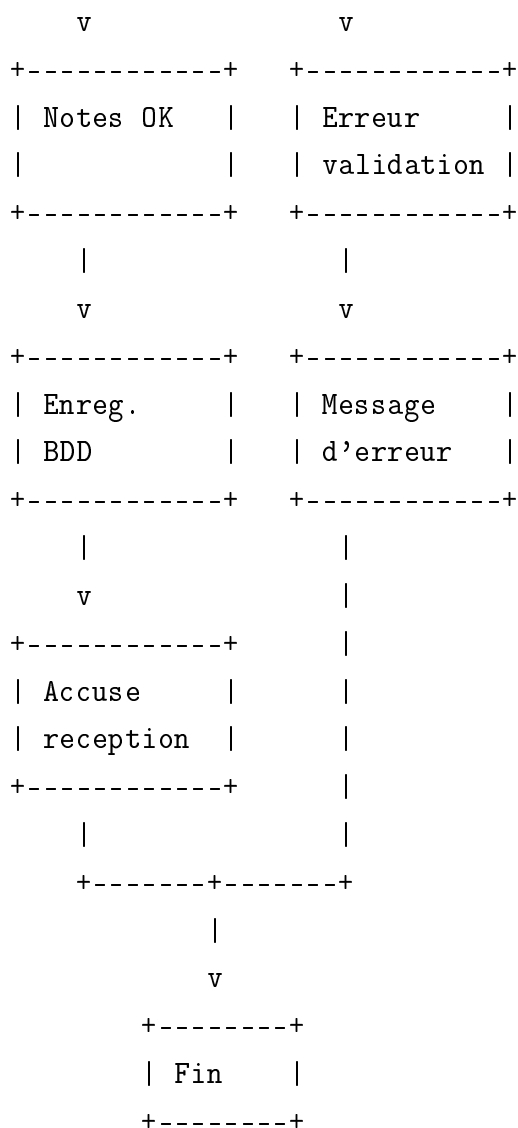


Figure III.10 : Diagramme d'activité – Saisie des notes

3.7 Modèle conceptuel de données

3.7.1 Diagramme de classe global

Le diagramme de classe présente les principales entités du système et leurs relations.

Entités principales :

1. **Ecole** : représente l'établissement scolaire
2. **Classe** : groupe d'élèves avec un niveau
3. **Élève** : apprenant inscrit dans une classe
4. **Professeur** : enseignant
5. **Matière** : discipline enseignée avec coefficient

6. **Note** : évaluation d'un élève dans une matière
7. **Bulletin** : relevé de notes périodique
8. **PresenceEleve** : enregistrement d'assiduité
9. **EmploiDuTemps** : planification des cours
10. **Parent** : représentant légal de l'élève
11. **Utilisateur** : personne ayant accès au système
12. **Permission** : droit d'accès à un module
13. **Paie** : transaction financière
14. **Message** : communication entre utilisateurs
15. **Conversation** : groupe de discussion
16. **Événement** : événement du calendrier scolaire
17. **Staff** : personnel non enseignant
18. **Département** : unité organisationnelle
19. **SyncDevice** : appareil connecté pour synchronisation

3.7.2 Relations principales

- **Ecole** 1 — N **Classe**
- **Classe** 1 — N **Élève**
- **Classe** N — N **Matière** (via **ClasseMatière**)
- **Professeur** N — N **Classe** (via **ProfesseurClasse**)
- **Élève** 1 — N **Note**
- **Élève** 1 — N **Bulletin**
- **Parent** N — N **Élève** (via **ParentEleve**)
- **Conversation** 1 — N **Message**
- **Élève** 1 — N **Paie**
- **Professeur** 1 — N **TeacherLesson**

3.7.3 Schéma relationnel simplifié

```

ECOLE (id, nom, logo)
|
+-- CLASSE (id, nom, niveau)
    |
    +-- ELEVE (id, matricule, nom, prenom, date_naissance,
```

```
|          adresse, telephone, genre, statut, id_classe)
|
+-- CLASSE_MATIERE (id_classe, id_matiere, coefficient)
|
+-- EMPLOI_DU_TEMPS (id, jour, heure_debut, heure_fin,
|                   id_classe, id_matiere, id_professeur)
|
+-- PRESENCE_ELEVE (id, date_jour, session_jour, statut,
                   eleve_id, classe_id, annee_scolaire)

PROFESSEUR (id, nom, prenom, email, telephone, specialisation)
|
+-- PROFESSEUR_CLASSE (id, professeur_id, classe_id, matiere_id)
|
+-- NOTE (id, valeur, trimestre, coefficient, eleve_id,
|         matiere_id, professeur_id)
|
+-- TEACHER_LESSON (id, titre, rubrique, date_prevue, statut)

MATIERE (id, nom, code)

BULLETIN (id, trimestre, moyenne, mention, appreciation, id_eleve)

PARENT (id, nom, prenom, lien, telephone, email)
|
+-- PARENT_ELEVE (parent_id, eleve_id)

UTILISATEUR (id, nom, email, mot_de_passe, role, avatar)
|
+-- PERMISSION (utilisateur_id, module, role, acces)

CONVERSATION (id, type, name, is_announcement)
|
+-- MESSAGE (id, conversation_id, sender_type, sender_id, content)

TYPE_PAIEMENT (id, nom, montant, mois, id_classe)
|
+-- PAIEMENT (id, type_id, personne_id, montant, mois, statut)
```

```
STAFF (id, nom, prenom, poste, departement_id, salaire_base)

SYNC_DEVICE (id, uuid, nom, type_appareil, role_sync, autorise)
|
+-- SYNC_BATCH (id, uuid, device_uuid, direction, statut)
    |
    +-- SYNC_CHANGE (id, uuid, batch_uuid, table_name,
                     record_uuid, operation, payload_json)
```

Figure III.11 : Schéma relationnel simplifié

3.8 Conclusion

Ce chapitre a présenté l'analyse et la conception du système Novaskol en utilisant le langage UML. Nous avons identifié 16 besoins fonctionnels et 10 besoins non fonctionnels, défini 7 acteurs avec leurs cas d'utilisation, modélisé les interactions dynamiques via des diagrammes de séquence et d'activité, et établi le modèle conceptuel de données avec toutes les entités et leurs relations.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la phase de réalisation de Novaskol. Nous décrivons en détail l'environnement de développement, l'implémentation du backend Laravel avec ses contrôleurs, services et middlewares, le frontend avec ses vues Blade, l'application de bureau Electron, les applications companion, le processus de packaging et de déploiement, et les interfaces principales de l'application.

4.2 Environnement de développement

4.2.1 Outils de développement

TABLE 4.1 – Outils de développement

Outil	Version	Utilisation
Visual Studio Code	Dernière	Éditeur de code principal
PHP	8.2.0	Langage serveur
Composer	2.x	Gestionnaire de dépendances PHP
Node.js	22.x	Exécution JavaScript
npm	10.x	Gestionnaire de paquets
Git	2.x	Contrôle de version
WampServer	3.x	Environnement PHP local (dev)
Electron	37.2.1	Framework desktop
electron-builder	26.0.12	Packaging d'installateur
Laravel Pint	Dernière	Correcteur de style

4.2.2 Structure du projet

novaskol-laravel/

```

+-- app/                                # Code source Laravel
|   +-- Console/Commands/              # Commandes Artisan
|   +-- Http/
|       +-- Controllers/               # 30 controleurs
|       +-- Middleware/                # 3 middlewares
|   +-- Models/                        # Modeles Eloquent
|   +-- Services/                      # 9 services metier
+-- apps/                              # Applications companion
|   +-- novaskol-connecte/             # Mobile React + Capacitor
|   +-- novaskol-connecte-desktop/     # Desktop companion Electron
+-- config/                            # Configuration Laravel
+-- database/                          # Base de donnees
|   +-- migrations/                   # 33 migrations
+-- desktop/                           # Application Electron principale
|   +-- main.cjs                      # Processus principal (220 lignes)
|   +-- preload.cjs                   # Pont IPC
+-- public/                            # Fichiers publics
+-- resources/                         # Vue Blade, CSS, JS
|   +-- views/                        # 66 templates Blade
+-- routes/                            # Definition des routes
|   +-- web.php                       # 287 lignes de routes
+-- tools/                             # Outils de build et runtime
|   +-- runtime/                      # PHP 8.2 portable + MySQL
|   +-- windows/                      # Scripts PowerShell
+-- vendor/                            # Dependances PHP

```

Figure IV.1 : Arborescence complète du projet

4.2.3 Processus de développement

Le développement a suivi une approche itérative avec des cycles de 2 semaines :

1. Phase d'analyse : Recueil des besoins, spécifications
2. Phase de conception : Modélisation UML, architecture technique
3. Phase d'implémentation : Codage des modules fonctionnels
4. Phase de test : Tests unitaires, tests d'intégration
5. Phase de validation : Tests utilisateurs, corrections
6. Phase de packaging : Build, déploiement, documentation

4.3 Implémentation du backend Laravel

4.3.1 Architecture des contrôleurs

Les contrôleurs sont organisés par domaine fonctionnel. Novaskol compte 30 fichiers de contrôleurs :

```
app/Http/Controllers/
+-- Auth/
|   +-- LegacyAuthController.php
+-- Dashboard/
|   +-- DashboardController.php
|   +-- RoleDashboardController.php
+-- Connected/
|   +-- ConnectedInitController.php
|   +-- ConnectedSyncController.php
+-- AccountingController.php    # Comptabilite
+-- BulletinController.php      # Bulletins
+-- ClassroomController.php     # Classes
+-- CommunicationController.php # Messagerie
+-- GradeController.php         # Notes
+-- StudentController.php       # Eleves
+-- SubjectController.php       # Matieres
+-- ScheduleController.php      # Emploi du temps
... (30 fichiers au total)
```

4.3.2 Système d'authentification

Le système d'authentification est basé sur les sessions PHP et la table utilisateurs :

```
// Verification des identifiants
$user = DB::table('utilisateurs')
    ->where('email', $request->email)
    ->where('role', $request->role)
    ->first();

if ($user && Hash::check($request->password, $user->password)) {
    session([
        'utilisateur_id'    => $user->id,
        'utilisateur_nom'   => $user->nom,
        'utilisateur_email' => $user->email,
```

```
        'utilisateur_role' => $user->role
    ]);
    return redirect('/dashboard');
}
```

4.3.3 Middleware de contrôle d'accès

Le middleware `module.access` est le mécanisme central de contrôle d'accès :

```
// Middleware module.access
$permission = DB::table('permissions')
    ->where('utilisateur_id', session('utilisateur_id'))
    ->where('module', $module)
    ->where('acces', true)
    ->exists();

if (!$permission) {
    abort(403, 'Acces non autorise');
}
```

Exemples d'utilisation dans les routes :

```
Route::get('/notes', [GradeController::class, 'index'])
    ->middleware('module.access:notes');

Route::get('/bulletin', [BulletinController::class, 'index'])
    ->middleware('module.access:bulletin');
```

4.3.4 Services métier

Les services métier encapsulent la logique complexe :

- **BulletinCalculator** : Calcul des moyennes, rang, mention
- **QrCodeService** : Génération de QR codes
- **ConnectedLocalSynchronizer** : Synchronisation bidirectionnelle
- **ModuleRegistry** : Enregistrement des modules
- **RelationalDeleteService** : Suppression en cascade
- **SyncService** : Gestion des lots de synchronisation
- **ExportService** : Export des données (Excel, PDF)

4.3.5 Génération de PDF

```
$pdf = Pdf::loadView('modules.professeur.bulletin.pdf', [  
    'eleve'      => $eleve,  
    'notes'      => $notes,  
    'moyennes'   => $moyennes,  
    'trimestre' => $trimestre  
]);  
return $pdf->download("bulletin-{$eleve->mat_etud}.pdf");
```

4.4 Implémentation du frontend

4.4.1 Architecture des vues

Le frontend utilise le moteur de templates Blade de Laravel avec 66 fichiers de vue organisés par module fonctionnel.

4.4.2 Design et thème

- **Thème sombre** par défaut avec mode clair activable
- **Palette de couleurs** : Fond sombre (#080e18), accent vert (#00c853)
- **Design responsive** grâce à TailwindCSS
- **Notifications** SweetAlert2 pour les interactions utilisateur

4.4.3 Composants d'interface

- Barre latérale : Navigation principale avec sous-menus
- En-tête fixe : Notifications, sélection langue, profil
- Tableau de bord : Widgets statistiques, graphiques Chart.js
- Tableaux de données : Listes avec recherche, tri, pagination
- Formulaire : Saisie avec validation client et serveur
- Calendrier : FullCalendar pour les événements
- Chat : Interface de messagerie avec réactions

4.4.4 Internationalisation

Le système supporte 6 langues : Français, Anglais, Allemand, Malgache, Espagnol, Portugais.

4.5 Application de bureau Electron

4.5.1 Fichiers clés

TABLE 4.2 – Fichiers clés de l'application Electron

Fichier	Taille	Rôle
desktop/main.cjs	220 lignes	Processus principal Electron
desktop/preload.cjs	16 lignes	Pont IPC sécurisé
desktop/ui/splash.html	135 lignes	Interface de démarrage
desktop/package.json	51 lignes	Configuration build

4.5.2 Processus principal (main.cjs)

```
async function startNovaskol(r) {
  // 1. Configurer l'environnement
  ensureLayout(r);
  configureEnv(r);
  ensureAppKey(r);

  // 2. Installer les dependances si necessaire
  if (!fs.existsSync(path.join(r, 'vendor'))) {
    runComposerInstall(r);
  }

  // 3. Executer les migrations
  runPhpArtisan(r, 'migrate --force');

  // 4. Demarrer le serveur PHP
  phpProcess = spawn(getPhp(r), [
    path.join(r, 'artisan'),
    'serve', '--host=0.0.0.0', '--port=8001'
  ], { cwd: r, windowsHide: true });

  // 5. Attendre que le serveur reponde
  await waitForHttp('http://127.0.0.1:8001', 120000);

  // 6. Ouvrir la fenetre principale
  mainWindow.loadURL('http://127.0.0.1:8001');
}
```

4.5.3 Fonctionnalités du processus principal

1. **Single-instance lock** : empêche le lancement multiple
2. **Résolution du runtime PHP** : 4 stratégies
3. **Fenêtre splash** : 460×360, frameless, thème sombre
4. **Fenêtre principale** : 1440×920 (min 1220×760)
5. **Configuration .env** automatique
6. **Migration forcée** au démarrage
7. **Health check** : Ping HTTP jusqu'à 120 secondes
8. **Fermeture** : Confirmation utilisateur, arrêt PHP

4.5.4 Pont IPC

```
contextBridge.exposeInMainWorld('desktopShell', {
  onSplashStatus(callback) {
    ipcRenderer.on('splash-status', (_event, message) =>
      callback(message));
  },
  async getMeta() {
    return ipcRenderer.invoke('desktop:get-meta');
  },
  print() {
    ipcRenderer.send('print');
  },
  printHtml(html) {
    ipcRenderer.send('print-html', html);
  }
});
```

4.5.5 Configuration electron-builder

```
{
  "build": {
    "appId": "com.novaskol.desktop",
    "productName": "Novaskol",
    "directories": {
      "output": "../storage/app/desktop-dist"
    },
    "extraResources": [{
```

```
        "from": "../storage/app/distribution/novaskol-app-latest",
        "to": "seed/novaskol",
        "filter": ["**/*"]
    }],
    "win": { "target": ["nsis"] },
    "nsis": {
        "oneClick": false,
        "allowToChangeInstallationDirectory": true,
        "createDesktopShortcut": true,
        "createStartMenuShortcut": true
    }
}
```

4.6 Application mobile companion (Novaskol Connecte)

4.6.1 Architecture

Novaskol Connecte est une application mobile hybride développée avec React 19, Vite 8, et Capacitor 8 pour Android.

Caractéristiques techniques :

- Version : 0.2.0
- Framework : React 19
- Build : Vite 8
- Natif : Capacitor 8.3.3 (Android)
- Offline : Service Worker + localStorage

4.6.2 Fonctionnalités

- Consultation des notes et bulletins
- Suivi des présences
- Messagerie
- Calendrier scolaire
- Gestion des élèves (CRUD)
- Calcul de bulletins côté client
- Synchronisation hors-ligne
- Appairage par QR code

4.7 Application de bureau Connecte Desktop

4.7.1 Architecture

Novaskol Connecte Desktop est une application Electron companion pour PC secondaire, embarquant son propre serveur PHP/Laravel.

Caractéristiques techniques :

- Version : 1.0.0
- Framework : Electron 37.2.1
- Backend : PHP 8.2 + Laravel embarqué
- Autosync : Toutes les 5 minutes

4.7.2 Différences avec la version mobile

TABLE 4.3 – Comparaison Connecte Mobile vs Desktop

Aspect	Connecte Mobile	Connecte Desktop
Technologie	React 19 + Capacitor	Electron + PHP embarqué
Backend	Externe (HTTP)	Local (PHP serveur)
Stockage	localStorage	SQLite locale
Sync	Push/Pull HTTP	PHP + sync automatique
Autonomie	Nécessite serveur principal	Autonome (seed Laravel)

4.8 Processus de packaging et déploiement

4.8.1 Application de bureau (Electron + electron-builder)

Le processus de build et packaging suit les étapes suivantes :

1. **Préparation** : `php artisan novaskol:prepare-installer-source -with-vendor`
2. **Copie du runtime** : PHP portable + MySQL portable
3. **Installation des dépendances npm** dans `desktop/`
4. **Build Electron** : `npm run build (electron-builder --win nsis)`
5. **Résultat** : Novaskol Setup 1.0.6.exe (~647 Mo)

4.8.2 PHP Runtime portable

- PHP 8.2.0 avec 19 extensions chargées
- `extension_dir="ext"` (chemin relatif)

- Opcache activé avec JIT tracing
- `memory_limit=128M`
- `max_execution_time=120`

4.8.3 Installateur NSIS

- Assistant d'installation graphique
- Choix du répertoire d'installation
- Raccourcis Bureau et Menu Démarrer
- Désinstallation propre

4.8.4 Modes de déploiement

TABLE 4.4 – Modes de déploiement

Mode	Base de données	Installation
Local (Desktop)	SQLite	Installateur NSIS
Hébergement	MySQL/MariaDB	Déploiement FTP/CLI
Hybride	SQLite + Sync réseau	Desktop + Connecte

4.9 Interfaces principales

4.9.1 Page d'authentification

- Logo et nom de l'application
- Formulaire de connexion avec sélection du rôle
- Fonctionnalités principales
- Options multilingues (6 langues)

4.9.2 Tableau de bord

- Widgets statistiques (élèves, enseignants, classes)
- Graphiques d'évolution (Chart.js)
- Événements du calendrier à venir
- Notifications récentes

4.9.3 Saisie des notes

- Sélection de la classe et de la matière
- Liste des élèves avec notes existantes
- Saisie avec validation automatique (0-20)
- Calcul automatique des moyennes

4.9.4 Génération de bulletins

- Calcul automatique des moyennes par matière
- Moyenne générale et rang
- Mention et appréciation automatiques
- Export PDF (via DOMPDF)

4.9.5 Emploi du temps

- Vue hebdomadaire par classe
- Affectation des matières et professeurs
- Intégration FullCalendar

4.9.6 Messagerie interne

- Conversations privées et de groupe
- Envoi de messages textes et fichiers
- Réactions par emojis
- Statut de saisie

4.9.7 Gestion des présences par QR code

- Génération de cartes d'identification avec QR code
- Scan de QR code
- Sessions matin et après-midi
- Statistiques de présence

4.10 Conclusion

Ce chapitre a présenté la réalisation complète de Novaskol. L'implémentation couvre l'ensemble des 45 modules fonctionnels à travers une architecture modulaire et évolutive.

Le backend Laravel avec ses 30 contrôleurs, 9 services et 3 middlewares constitue le cœur du système. Le frontend offre 66 vues Blade avec un design moderne et responsive. L'application de bureau Electron permet un déploiement simplifié et un fonctionnement complètement hors-ligne. Le packaging via electron-builder produit un installateur Windows autonome de 647 Mo.

Conclusion générale

Le travail présenté dans ce mémoire avait pour objectif la conception et la réalisation d'un système de gestion scolaire complet, fonctionnant hors-ligne, facile à déployer et à utiliser. Nous avons conçu et développé **Novaskol**, une application qui répond aux besoins spécifiques des établissements scolaires en matière de gestion administrative, pédagogique et financière.

Résultats obtenus

Au terme de ce projet, nous avons atteint les objectifs fixés :

1. **Un système complet** : 45 modules fonctionnels couvrant l'ensemble des besoins de gestion scolaire.
2. **Une architecture robuste** : Basée sur Laravel 12 et PHP 8.2 avec Eloquent ORM, architecture MVC.
3. **Une base de données exhaustive** : 42 tables avec 33 migrations Laravel.
4. **Une application de bureau autonome** : Electron 37, installateur NSIS de 647 Mo.
5. **Deux applications companion** : Novaskol Connecte (mobile) et Connecte Desktop.
6. **Une synchronisation multi-appareils** : Réseau local avec appairage et résolution de conflits.
7. **Une interface intuitive** : Adaptée à chaque profil d'utilisateur, design moderne.
8. **Un déploiement simplifié** : Installateur Windows unique contenant tous les composants.

Difficultés rencontrées

- Intégration de PHP dans Electron : gestion des processus fils, chemins d'accès
- Packaging du runtime PHP : configuration du php.ini avec chemins relatifs
- Synchronisation multi-appareils : UUIDs globaux, détection de conflits

- Migration du code legacy : base MySQL `bulletin_system` vers Laravel
- Gestion des chemins dans `php.ini` : portabilité entre machines

Compétences acquises

Ce projet m’a permis d’acquérir et de renforcer des compétences clés, en prolongement de mon parcours CAN et de ma spécialisation web :

- Développement web full-stack : Laravel (MVC, Eloquent ORM, migrations, Blade), PHP orienté objet, React 19
- Applications de bureau hybrides : Electron, IPC, gestion de processus, packaging via electron-builder
- Applications mobiles : React 19, Capacitor 8, PWA, IndexedDB, synchronisation hors-ligne
- Bases de données : SQLite, MySQL/MariaDB, conception de schémas, optimisation de requêtes
- Modélisation et conception : UML (cas d’utilisation, diagrammes de séquence, d’activité, de classes)
- Packaging et déploiement : electron-builder, NSIS, configuration d’installateur, runtime PHP portable
- Scripting et automatisation : PowerShell, gestion des processus, configuration système
- Gestion de projet : Planification, méthodologie agile, documentation, versionnement Git
- Intégration des technologies : capacité à assembler des briques technologiques diverses (PHP, Node.js, React, SQLite, Electron) en un produit cohérent et fonctionnel

Perspectives d’évolution

1. Application mobile native (iOS/Android) avec React Native ou Flutter
2. Version cloud SaaS avec hébergement multi-établissements
3. Synchronisation Internet avec chiffrement de bout en bout
4. Intelligence artificielle pour la prédiction des résultats scolaires
5. API REST publique documentée pour l’intégration avec des systèmes tiers
6. Modules complémentaires : bibliothèque, restauration, transport

7. Compatibilité multi-plateforme (macOS, Linux)
8. Tableaux de bord avancés avec analytics
9. Mode multi-établissement
10. Signature électronique pour les documents officiels

Mot de la fin

La réalisation de ce projet a été pour moi bien plus qu'un exercice académique. Issu d'un parcours en Communication en Audiovisuelle et Numérique (CAN), je me suis tourné vers le web après la L3, animé par l'envie de comprendre et de maîtriser les technologies qui façonnent notre monde numérique. Novaskol représente l'aboutissement de cette transition : un système complet, du dossier de conception à l'installateur final, en passant par la base de données, le code et l'interface utilisateur.

J'espère que cet outil contribuera à améliorer la gestion et la qualité de l'enseignement dans les établissements qui l'adopteront. Mais au-delà de l'outil lui-même, ce projet restera pour moi la trace tangible d'un parcours, d'un choix et d'une volonté de laisser une empreinte dans le domaine qui me passionne : le web.

Novaskol est un projet open source (licence MIT), dont le code source est librement accessible et modifiable. La communauté est encouragée à contribuer à son amélioration continue.

Bibliographie

Ouvrages et articles

- [1] Kruchten, P. (1995). "4+1 View Model of Software Architecture." *IEEE Software*, 12(6), 42-50.
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [3] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide*. 2nd Edition. Addison-Wesley.
- [4] Roques, P. (2004). *UML par la pratique*. 2e édition. Éditions Eyrolles.
- [5] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- [6] Gardarin, G. (2000). *Internet/Intranet et les bases de données*. Éditions Eyrolles.
- [7] Comer, D. (2003). *Internet : Services et réseaux*. Éditions Dunod.
- [8] Otlet, J. (2002). *Architectures de Systèmes d'Information*. Livre Blanc OCTO.
- [9] Ullman, L. (2023). *PHP for the Web : Visual QuickStart Guide*. 5th Edition. Peachpit Press.

Documentation technique

- [10] Laravel Documentation. *Laravel 12.x Documentation*. <https://laravel.com/docs/12.x>
- [11] PHP Documentation. *PHP Manual*. <https://www.php.net/docs.php>
- [12] Electron Documentation. *Electron Documentation*. <https://www.electronjs.org/docs>
- [13] electron-builder Documentation. *electron-builder Documentation*. <https://www.electron.build/>
- [14] SQLite Documentation. *SQLite Documentation*. <https://www.sqlite.org/docs.html>

- [15] TailwindCSS Documentation. *TailwindCSS v4 Documentation*. <https://tailwindcss.com/docs>
- [16] React Documentation. *React 19 Documentation*. <https://react.dev/>
- [17] Capacitor Documentation. *Capacitor 8 Documentation*. <https://capacitorjs.com/docs>
- [18] MySQL Documentation. *MySQL 8.0 Documentation*. <https://dev.mysql.com/doc/>
- [19] DOMPDF. *Laravel DOMPDF*. <https://github.com/barryvdh/laravel-dompdf>
- [20] PhpSpreadsheet. *PhpSpreadsheet Documentation*. <https://phpspreadsheet.readthedocs.io/>

Références web

- [21] Wikipedia. “Architecture logicielle.” https://fr.wikipedia.org/wiki/Architecture_logicielle
- [22] Wikipedia. “Client-serveur.” <https://fr.wikipedia.org/wiki/Client-serveur>
- [23] Wikipedia. “PHP.” <https://fr.wikipedia.org/wiki/PHP>
- [24] Wikipedia. “Laravel.” <https://fr.wikipedia.org/wiki/Laravel>
- [25] Wikipedia. “Electron.” [https://fr.wikipedia.org/wiki/Electron_\(framework\)](https://fr.wikipedia.org/wiki/Electron_(framework))
- [26] Wikipedia. “UML.” [https://fr.wikipedia.org/wiki/UML_\(informatique\)](https://fr.wikipedia.org/wiki/UML_(informatique))
- [27] Wikipedia. “SQLite.” <https://fr.wikipedia.org/wiki/SQLite>
- [28] Wikipedia. “Modèle-Vue-Contrôleur.” <https://fr.wikipedia.org/wiki/ModÃ©le-Vue-ContrÃªleur>

Annexe A

Généralités sur les réseaux informatiques

A.1 Définition d'un réseau

Un réseau informatique est un ensemble d'équipements informatiques reliés entre eux pour échanger des informations. Les réseaux permettent le partage de ressources, la communication entre utilisateurs, et l'accès à des services centralisés.

A.2 Intérêts d'un réseau informatique

- Partage de fichiers et de périphériques
- Communication entre utilisateurs
- Centralisation des données
- Accès à Internet
- Travail collaboratif

A.3 Classification des réseaux

- **LAN (Local Area Network)** : Réseau local, couvre une zone limitée
- **MAN (Metropolitan Area Network)** : Réseau métropolitain
- **WAN (Wide Area Network)** : Réseau étendu

A.4 Topologies de réseau

- Topologie en bus
- Topologie en anneau
- Topologie en étoile (la plus courante)

A.5 Modèle OSI

Le modèle OSI (Open Systems Interconnection) définit 7 couches de communication :

1. Physique : Transmission des bits sur le support
2. Liaison de données : Accès au support, détection d'erreurs
3. Réseau : Routage des paquets (IP)
4. Transport : Transport fiable des données (TCP)
5. Session : Gestion des sessions de communication
6. Présentation : Codage et décodage des données
7. Application : Interfaces avec les applications utilisateur (HTTP, FTP, SMTP)

A.6 Protocole TCP/IP

TCP/IP est le protocole fondamental d'Internet. Il combine :

- **IP (Internet Protocol)** : Routage des paquets entre les machines
- **TCP (Transmission Control Protocol)** : Transport fiable avec contrôle de flux

Annexe B

UML – Unified Modeling Language

B.1 Définition

UML (Unified Modeling Language) est un langage de modélisation graphique utilisé en génie logiciel pour visualiser, spécifier, construire et documenter les systèmes d'information.

B.2 Diagrammes UML utilisés dans Novaskol

Diagramme de cas d'utilisation : Représente les fonctionnalités du système du point de vue des acteurs externes.

Diagramme de séquence : Montre les interactions entre objets au cours du temps.

Diagramme d'activité : Représente les flux de contrôle et de données.

Diagramme de classes : Montre la structure statique du système.

B.3 Éléments d'UML

- **Acteur** : Entité qui interagit avec le système
- **Cas d'utilisation** : Fonctionnalité offerte par le système
- **Classe** : Abstraction d'une entité du domaine
- **Relation** : Association, inheritance, dépendance, réalisation
- **Message** : Communication entre objets
- **État** : Condition dans laquelle se trouve un objet

Annexe C

Guide d'installation de Novaskol

C.1 Installation locale (Desktop)

1. Télécharger `Novaskol-Setup-1.0.6.exe` (647 Mo)
2. Exécuter l'installateur
3. Suivre l'assistant d'installation
4. Lancer Novaskol depuis le raccourci Bureau ou Menu Démarrer
5. Au premier démarrage, l'application initialise automatiquement la base de données
6. Créer un compte administrateur
7. Configurer l'établissement

C.2 Configuration minimale requise

- Système d'exploitation : Windows 10/11 64-bit
- Processeur : Intel Core i3 ou équivalent
- RAM : 4 Go minimum (8 Go recommandé)
- Espace disque : 2 Go disponibles
- Aucune connexion Internet requise

C.3 Installation sur hébergement web

1. Déployer les fichiers du projet sur le serveur web
2. Configurer le fichier `.env`
3. Exécuter `php artisan migrate -force`
4. Exécuter `php artisan key:generate`
5. Configurer le serveur web (Apache/Nginx)
6. Créer un compte administrateur

Annexe D

Structure du paquet de distribution

Le paquet de distribution `novaskol-app-latest/` contient l'ensemble des fichiers nécessaires :

```
novaskol-app-latest/
+-- app/                # Code Laravel
+-- bootstrap/          # Bootstrap Laravel
+-- config/             # Fichiers de configuration
+-- database/           # Migrations et dumps
+-- public/             # Point d'entree web
+-- resources/          # Vues Blade et assets
+-- routes/             # Routes de l'application
+-- storage/            # Stockage (logs, cache)
+-- tools/              # Outils
|   +-- runtime/
|       +-- php/        # PHP 8.2.0 portable
|       +-- mysql/      # MySQL 8.0.31 portable
+-- vendor/            # Dependances PHP (Composer)
+-- main.cjs           # Processus principal Electron
+-- preload.cjs        # Pont IPC
+-- composer.json      # Dependances PHP
+-- package.json       # Dependances Node.js
```

Annexe E

Tableau des routes API

Points d'accès API REST pour la synchronisation des appareils connectés :

TABLE E.1: Routes API de synchronisation

Méthode	Route	Description
GET	/reseau-local/ping	Test de connexion au serveur
GET	/reseau-local/manifest-appareil	Manifeste du serveur
POST	/reseau-local/appairer-appareil	Appairage d'un nouvel appareil
GET	/reseau-local/bootstrap-appareil	Données initiales
POST	/reseau-local/recevoir-lot	Réception d'un lot de modifications
GET	/reseau-local/appareil-connecte	Page d'appairage
OPTIONS	/reseau-local/any	Preflight CORS
GET	/connected/init	Initialisation appareil connecté
GET	/connected/sync/status	Statut de la synchronisation
POST	/connected/sync/run	Exécution de la synchronisation
POST	/connected/disconnect	Déconnexion de l'appareil
POST	/connected/switch-user	Changement d'utilisateur connecté